
XAIMetrics

Maria Teresa Alba Rueda

Sep 21, 2026

CONTENTS

1	Base	1
2	Configuration and execution	7
3	Explainers	15
4	Metrics	21
4.1	Complexity	21
4.2	Faithfulness	23
4.3	Fidelity	31
4.4	Robustness	47
4.5	Sensitivity	56
	Python Module Index	63
	Index	65

class xai_metrics.base.base_metric.**BaseMetric**(*context, params=None*)

Bases: object

Base class for all metric implementations.

Metric classes should inherit from this class, define their own **NAME**, and implement the `run()` method.

Parameters

- **context** (**MetricContext**) – Shared context containing the model, test data, labels, observations, attribution values and optional device information.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific configuration parameters. If *None*, an empty dictionary is used.

run()

Execute the metric computation.

Subclasses must override this method with the actual metric logic.

Returns

Metric result returned by the concrete implementation.

Return type

Any

Raises

NotImplementedError – If the subclass does not implement this method.

class xai_metrics.base.base_metric.**MetricContext**(*model, X_test, y_test, observations, attributions, device=None*)

Bases: object

Shared context used by metric implementations.

This dataclass stores the model, test data, labels, selected observations, attribution values, and optional extra information required by metric classes.

Parameters

- **model** (*torch.nn.Module*) – Model evaluated by the metrics.
- **X_test** (*pandas.DataFrame*) – Test input data used for metric computation.
- **y_test** (*pandas.Series*) – Test labels associated with **X_test**.
- **observations** (*Any*) – Identifiers of the observations explained by the attribution matrix. These values usually correspond to indexes in **X_test** and **y_test**.

- **attributions** (*numpy.ndarray*) – Attribution values for the selected observations. Each row usually corresponds to one observation and each column to one feature.
- **device** (*str* or *None*) – Device where the model is placed, such as "cpu" or "cuda". If *None*, no explicit device was configured.

exception `xai_metrics.base.base_metric.MetricSkipped`

Bases: `Exception`

Exception raised when a metric cannot be computed.

This exception should be used when a metric is not applicable to the current context, model, data, or attribution values.

class `xai_metrics.base.base_explainer.BaseExplainer(context, params=None)`

Bases: `object`

Base class for explanation methods.

Explainer classes should inherit from this class, define their own `NAME` and implement `explain()`, which generates attribution values for a batch of inputs.

Instances are callable, so calling an explainer is equivalent to calling `explain()`. They can also expose their explanation method through `as_explain_func()`, which returns a metric-compatible function for metrics that need to recompute explanations on perturbed inputs.

Parameters

- **context** (`ExplainerContext`) – Shared context containing the background data and optional model, batch, labels and device information used by the explainer.
- **params** (`Mapping[str, Any]` or *None*, optional) – Explainer-specific configuration parameters. A copy of the mapping is stored in `params`. If *None*, an empty dictionary is used.

as_explain_func()

Return the explanation method as a metric-compatible function.

The returned bound method can be passed as an `explain_func` dependency to metrics that need to generate explanations for modified or perturbed inputs.

Returns

Bound `explain()` method of the current explainer instance.

Return type

`ExplainFunc`

explain(*model*, *inputs*, *targets=None*, ***kwargs*)

Generate explanations for the provided inputs.

Subclasses must override this method with the logic required by the corresponding explanation method.

Parameters

- **model** (`torch.nn.Module`) – Model whose predictions are to be explained.
- **inputs** (*Any*) – Input observation or batch of observations to explain.
- **targets** (*Any* or *None*, optional) – Target outputs, classes or labels for which explanations are generated. Their interpretation depends on the explainer.
- ****kwargs** (*Any*) – Additional arguments required by the concrete explanation method.

Returns

Attribution values generated for the provided inputs.

Return type

numpy.ndarray

Raises

NotImplementedError – Always raised by the base implementation. Subclasses must override this method.

```
class xai_metrics.base.base_explainer.ExplainerContext(X_background, y_background=None,  
                                                    model=None, X_batch=None,  
                                                    y_batch=None, device=None)
```

Bases: object

Shared context used by explainer implementations.

This dataclass stores the background data required by explanation methods and, optionally, the model, current batch, target values and device used during explanation generation.

Parameters

- **X_background** (*pandas.DataFrame*) – Background or reference input data used by the explainer.
- **y_background** (*pandas.Series or None, optional*) – Labels or targets associated with X_background. This is only required by explainers that need background target values.
- **model** (*Any or None, optional*) – Model associated with the explainer context. This can be used by explainers that store the model in the context instead of receiving it only at call time.
- **X_batch** (*pandas.DataFrame or None, optional*) – Current batch of observations to be explained.
- **y_batch** (*pandas.Series or None, optional*) – Labels or targets associated with X_batch.
- **device** (*str or None, optional*) – Device used for model execution, such as "cpu" or "cuda".

```
exception xai_metrics.base.base_explainer.ExplainerSkipped
```

Bases: Exception

Exception raised when an explainer cannot generate attributions.

This exception should be used when an explainer is not applicable to the current context, model, data, targets, or configuration.

```
xai_metrics.base.metric_registry.build_metrics_from_config(metrics_cfg, context,  
                                                         dependencies=None)
```

Build metric instances from a configuration list.

Each metric configuration must contain a **name** field matching a registered metric. It may also contain a **params** field with metric-specific parameters. Additional dependencies are injected only if their names are accepted by the metric class constructor.

Parameters

- **metrics_cfg** (*List[Mapping[str, Any]]*) – List of metric configuration dictionaries. Each dictionary must contain the metric **name** and may contain **params**.
- **context** (*MetricContext*) – Shared metric evaluation context passed to every metric instance.

- **dependencies** (*Mapping[str, Any] or None, optional*) – Optional dependency objects that can be injected into metric constructors. Only dependencies whose names appear in the constructor signature are passed.

Returns

List of instantiated metric objects.

Return type

List[*BaseMetric*]

Raises

- **ValueError** – If a metric name from `metrics_cfg` is not registered.
- **KeyError** – If a metric configuration does not contain the required `name` field.

`xai_metrics.base.metric_registry.list_metrics()`

Return the names of the registered metrics.

Returns

Sorted list containing the names of all metrics currently registered in `METRIC_REGISTRY`.

Return type

List[str]

`xai_metrics.base.metric_registry.register_metric(cls)`

Register a metric class in the global metric registry.

The metric is registered using its `NAME` attribute. If the class does not define `NAME`, the class name is used instead.

Parameters

cls (*Type[BaseMetric]*) – Metric class to register.

Returns

The same metric class passed as input. This allows the function to be used as a decorator.

Return type

Type[*BaseMetric*]

Raises

ValueError – If another metric with the same name is already registered.

Examples

```
>>> @register_metric
... class MyMetric(BaseMetric):
...     NAME = "my_metric"
...     def run(self):
...         return 0.0
```

`xai_metrics.base.explainer_registry.build_explainers_from_config(explainers_cfg, context)`

Build explainer instances from a configuration list.

Each explainer configuration must contain a `name` field matching a registered explainer. It may also contain a `params` field with explainer-specific parameters.

Parameters

- **explainers_cfg** (*List[Mapping[str, Any]]*) – List of explainer configuration dictionaries. Each dictionary must contain the explainer `name` and may contain `params`.

- **context** ([ExplainerContext](#)) – Shared explainer context passed to every explainer instance.

Returns

List of instantiated explainer objects.

Return type

List[[BaseExplainer](#)]

Raises

- **ValueError** – If an explainer name from `explainers_cfg` is not registered.
- **KeyError** – If an explainer configuration does not contain the required name field.

`xai_metrics.base.explainer_registry.list_explainers()`

Return the names of the registered explainers.

Returns

Sorted list containing the names of all explainers currently registered in `EXPLAINER_REGISTRY`.

Return type

List[str]

`xai_metrics.base.explainer_registry.register_explainer(cls)`

Register an explainer class in the global explainer registry.

The explainer is registered using its `NAME` attribute. If the class does not define `NAME`, the class name is used instead.

Parameters

cls (Type[[BaseExplainer](#)]) – Explainer class to register.

Returns

The same explainer class passed as input. This allows the function to be used as a decorator.

Return type

Type[[BaseExplainer](#)]

Raises

ValueError – If another explainer with the same name is already registered.

Examples

```
>>> @register_explainer
... class MyExplainer(BaseExplainer):
...     NAME = "my_explainer"
...     def explain(self, model, inputs, targets=None, **kwargs):
...         return inputs
```


CONFIGURATION AND EXECUTION

```
class xai_metrics.config.config_controller.ConfigController(config='config.yaml',  
                                                         model_loader=None)
```

Controller for loading configuration files and building contexts.

The controller can load configuration data from a mapping or from a YAML file. It supports direct context definitions and automatic discovery from dataset, model and attribution directories.

Metric contexts include models, test data, labels and precomputed attributions. Explainer contexts include models, background data and, optionally, batches to explain.

Parameters

- **config** (*Mapping[str, Any], str or pathlib.Path, default="config.yaml"*)
– Configuration source. If a mapping is provided, it is copied directly. If a path is provided, the YAML file is loaded. The default value loads the `config.yaml` file located next to this module.
- **model_loader** (*Callable[[str or pathlib.Path], Any] or None, optional*)
– Function used to load models from disk. The function must receive a model path and return the loaded model object. If `None`, `default_model_loader()` is used.

```
build_explainers_context(ctx_cfg=None)
```

Build an explainer context.

The context configuration must contain a model path and background input data. It may also contain background labels, a batch of inputs to explain, batch labels and a device. The method loads these objects, validates batch indexes when labels are available and returns both the `ExplainerContext` and its metadata.

Parameters

ctx_cfg (*Mapping[str, Any] or None, optional*) – Context configuration to use. If `None`, the `context` section of the loaded configuration is used. The configuration must contain `model_path` and `X_background_path`. It must also contain `dataset_name` and `model_name`. It may optionally contain `y_background_path`, `X_batch_path`, `X_test_path`, `y_batch_path`, `y_test_path` and `device`.

Returns

Explainer context and metadata dictionary.

Return type

Tuple[`ExplainerContext`, Dict[str, Any]]

Raises

- **ValueError** – If the `context` section is missing, required fields are missing, metadata are incomplete or batch indexes are inconsistent.

- **RuntimeError** – If a CUDA device is requested but CUDA is not available.

Warns

UserWarning – If a target file contains more than one column. In that case, only the first column is used.

build_explainers_contexts()

Build all explainer contexts defined by the configuration.

The method first obtains all explainer context configurations through `_iter_explainer_context_configs()` and then builds each corresponding `ExplainerContext`.

Returns

List of explainer contexts together with their metadata.

Return type

List[Tuple[[*ExplainerContext*](#), Dict[str, Any]]]

build_metric_context(*ctx_cfg=None*)

Build a metric evaluation context.

The context configuration must contain paths to the model, input data, labels and attribution file. The method loads these objects, validates indexes, converts attribution values to a NumPy array, optionally moves the model to the configured device and returns both the `MetricContext` and its metadata.

Parameters

ctx_cfg (*Mapping[str, Any] or None, optional*) – Context configuration to use. If `None`, the `context` section of the loaded configuration is used. The configuration must contain `model_path`, `X_test_path`, `y_test_path` and `attributions_path`. It must also contain `dataset_name`, `model_name` and `xai_method_name`. It may optionally contain `device`.

Returns

Metric context and metadata dictionary.

Return type

Tuple[[*MetricContext*](#), Dict[str, Any]]

Raises

- **ValueError** – If the `context` section is missing, required paths are missing, metadata are incomplete or indexes are inconsistent.
- **TypeError** – If the loaded model is not an instance of `torch.nn.Module`.
- **RuntimeError** – If a CUDA device is requested but CUDA is not available.

Warns

UserWarning – If `y_test` contains more than one column. In that case, only the first column is used.

build_metric_contexts()

Build all metric evaluation contexts defined by the configuration.

The method first obtains all metric context configurations through `_iter_metric_context_configs()` and then builds each corresponding `MetricContext`.

Returns

List of metric contexts together with their metadata.

Return type

List[Tuple[[*MetricContext*](#), Dict[str, Any]]]

get_explainers_config()

Return the explainers configuration section.

Returns

List of explainer configuration dictionaries. If the section is not defined, an empty list is returned.

Return type

List

get_metrics_config()

Return the metrics configuration section.

Returns

List of metric configuration dictionaries. If the section is not defined, an empty list is returned.

Return type

List

xai_metrics.config.config_controller.default_model_loader(model_path)

Load a model from disk.

Files with `.pkl` or `.pickle` extensions are loaded with `pickle`. Files with `.joblib` or `.jl` extensions are loaded with `joblib`. Any other file extension is loaded with `torch.load()` using `weights_only=False`.

Parameters

model_path (*str or pathlib.Path*) – Path to the saved model file.

Returns

Loaded model object.

Return type

Any

xai_metrics.runner.runner.run_evaluation(context=None, metadata=None, selected_metrics=None, config='config.yaml', report_output_dir=WindowsPath('results/reports'), **runtime_kwargs)

Run the XAI metric evaluation pipeline.

The function discovers all metric modules, loads the experiment configuration, builds or accepts one or more metric contexts, resolves the metrics that can be executed, injects runtime dependencies and evaluates each selected metric.

When `context` is `None`, metric contexts are built from `config` through `ConfigController`. When a context is supplied directly, `metadata` must also be provided.

Runtime keyword arguments are forwarded as dependencies to metric constructors when their names match constructor parameters. The `model_loader` argument is passed to the configuration controller.

The special `explain_funcs` argument may contain either a global mapping from XAI method names to explanation functions or a nested mapping from dataset names to XAI method names. Names are matched case-insensitively. The function corresponding to the current context is injected into compatible metrics under the `explain_func` dependency.

Metric results are stored as a list of dictionaries. Each result entry contains the metric name, the metric parameters used in that execution and the returned value. Metrics that raise `MetricSkipped` are recorded in `skipped` with their parameters and skip reason, without stopping the remaining metrics.

Summary reports and observation-level reports are built from the collected outputs. If `report_output_dir` is not `None`, reports are saved to disk after metric results become available.

Parameters

- **context** (*MetricContext* or *None*, optional) – Pre-built metric evaluation context. If *None*, one or more contexts are constructed from *config*.
- **metadata** (*Mapping[str, Any]* or *None*, optional) – Metadata associated with a directly supplied context. It must contain *dataset_name*, *model_name* and *xai_method_name*. Ignored when contexts are built from the configuration.
- **selected_metrics** (*Iterable[str]* or *None*, optional) – Metric names to execute. Only metrics that are also configured and registered are evaluated. If *None*, all configured and registered metrics are evaluated.
- **config** (*Mapping[str, Any]*, *str* or *pathlib.Path*, default="config.yaml") – Experiment configuration source. It may be a mapping or the path to a YAML configuration file.
- **report_output_dir** (*str*, *pathlib.Path* or *None*, optional) – Directory where the generated reports are saved. If *None*, reports are built but not written to disk. The default directory is *results/reports*.
- ****runtime_kwargs** (*Any*) – Runtime dependencies made available to metric constructors. Supported values mostly depend on the selected metrics.

Two arguments receive special handling:

- **model_loader** : Callable, optional Function used by the configuration controller to load models.
- **explain_funcs** : *Mapping[str, Callable]* or *Mapping[str, Mapping[str, Callable]]*, optional Mapping from XAI method names to explanation functions, or nested mapping from dataset names to XAI method names.

Returns

Dictionary containing the following entries:

- **"contexts"** : *List[Dict[str, Any]]* Raw output for each evaluated context. Every context output contains *metadata*, *observations*, *results* and *skipped*. *results* is a list of dictionaries with *metric*, *metric_params* and *value*. *skipped* is a list of dictionaries with *metric*, *metric_params* and *reason*.
- **"reports"** : *Dict[str, Dict[str, pandas.DataFrame]]* Summary report dataframes grouped first by dataset name and then by model name.
- **"observation_reports"** : *Dict[str, Dict[str, Dict[str, pandas.DataFrame]]]* Observation-level reports grouped by metric, dataset and model.
- **"report_paths"** : *Dict[str, Any]* Paths of the saved report files. The dictionary is empty when *report_output_dir* is *None*.

Return type

Dict[str, Any]

Raises

- **ValueError** – If metadata required for a directly supplied context is missing, if no explanation function is available for the current XAI method, or if the configuration or report data are invalid.
- **TypeError** – If context construction loads an object that is not a supported model type.
- **RuntimeError** – If context construction requests a device that is not available.

Warns

UserWarning – If configured or selected metrics cannot be executed because they are missing from the configuration or metric registry.

```
xai_metrics.runner.runner.run_explanation(context=None, selected_explainers=None,
                                         config='config.yaml',
                                         attribution_output_dir=WindowsPath('results/attributions'),
                                         **runtime_kwargs)
```

Run the XAI explanation generation pipeline.

The function discovers all explainer modules, loads the experiment configuration, builds or accepts one or more explainer contexts, resolves the explainers that can be executed and generates attribution values for each selected explainer.

When `context` is `None`, explainer contexts are built from `config` through `ConfigController`. When a context is supplied directly, the configuration must provide `dataset_name` and `model_name` in its `context` section.

Generated attributions are stored together with the parameters configured for each explainer. Explainers that raise `ExplainerSkipped` are recorded separately and do not stop the remaining explainers.

If `attribution_output_dir` is not `None`, generated attribution matrices are also saved as CSV files.

Parameters

- **context** (`ExplainerContext` or `None`, optional) – Pre-built explainer context. If `None`, one or more contexts are constructed from `config`.
- **selected_explainers** (`Iterable[str]` or `None`, optional) – Explainer names to execute. Only explainers that are also configured and registered are run. If `None`, all configured and registered explainers are executed.
- **config** (`Mapping[str, Any]`, `str` or `pathlib.Path`, default="config.yaml") – Experiment configuration source. It may be a mapping or the path to a YAML configuration file.
- **attribution_output_dir** (`str`, `pathlib.Path` or `None`, optional) – Directory where generated attribution CSV files are saved. If `None`, attributions are returned but not written to disk. The default directory is `results/attributions`.
- ****runtime_kwargs** (`Any`) – Runtime objects used by the explanation workflow. The `model_loader` argument receives special handling and is passed to `ConfigController`.

Returns

Dictionary containing the following entries:

- **"contexts"** : `List[Dict[str, Any]]` Raw output for each explainer context. Each entry contains `metadata`, `attributions`, `explainer_params` and `skipped`.
- **"attribution_paths"** : `Dict[str, str]` Paths of the saved attribution files. The dictionary is empty when `attribution_output_dir` is `None`.

Return type

`Dict[str, Any]`

Raises

- **ValueError** – If the configuration has no `context` section, if required metadata are missing, or if an `ExplainerContext` does not include a model or `X_batch`.
- **RuntimeError** – If context construction requests a device that is not available.

Warns

UserWarning – If configured or selected explainers cannot be executed because they are missing from the configuration or explainer registry.

`xai_metrics.reporting.reporting.build_observation_reports(context_outputs)`

Build observation-level metric reports from context outputs.

Observation-level reports preserve one metric value per evaluated observation instead of aggregating metric outputs. Reports are grouped by metric, dataset and model. Each dataframe contains one row per observation and one column per XAI method.

Metric outputs that cannot be aligned with the observation list are stored as `None` for the corresponding XAI method. Different parameter configurations for the same metric are kept as separate rows because the grouping includes the serialized `metric_params` signature.

Parameters

context_outputs (*Sequence[Mapping[str, Any]]*) – Sequence of context output mappings. Each element must contain:

- **metadata** : Mapping[str, Any] Metadata identifying the dataset, model and XAI method.
- **results** : Sequence[Mapping[str, Any]] Metric result entries. Each entry must contain **metric** and **value** and may contain **metric_params**.
- **observations** : Sequence[Any] Observation identifiers corresponding to the metric outputs.

Returns

Nested report dictionary. The first level is indexed by metric name, the second by dataset name and the third by model name.

Return type

Dict[str, Dict[str, Dict[str, pandas.DataFrame]]]

Raises

- **ValueError** – If a context output does not include **observations**, if observation indexes differ across XAI methods for the same report, or if a duplicated observation-level result is found for the same metric, parameter configuration and XAI method.
- **KeyError** – If required metadata or metric result fields are missing.

`xai_metrics.reporting.reporting.build_reports(context_outputs)`

Build summary metric reports from context outputs.

Each context output must contain a **metadata** mapping and a **results** sequence. Metadata must define **dataset_name**, **model_name** and **xai_method_name**. Each metric result in **results** must contain the fields **metric** and **value** and may contain **metric_params**.

Results are grouped by metric, dataset, model and metric parameters. Each generated dataframe contains the fixed columns **dataset_name**, **model_name**, **metric** and **metric_params**, followed by one result column for each XAI method.

Numeric metric values are flattened, `nan` values are removed and the remaining values are averaged. Non-numeric values are serialized without aggregation. If a metric value is a mapping, each entry is expanded into a separate report row named "<metric_name>.<result_key>".

Parameters

context_outputs (*Sequence[Mapping[str, Any]]*) – Sequence of context output mappings. Each element must contain:

- **metadata** : Mapping[str, Any] Metadata identifying the dataset, model and XAI method.

- **results** : Sequence[Mapping[str, Any]] Metric result entries. Each entry must contain **metric** and **value** and may contain **metric_params**.

Returns

Nested report dictionary. The first level is indexed by dataset name and the second level by model name. Each dataframe has one row per report metric and parameter configuration, and one result column per XAI method.

Return type

Dict[str, Dict[str, pandas.DataFrame]]

Raises

- **KeyError** – If a context output or metric result does not contain the required fields.
- **ValueError** – If an XAI method name conflicts with a reserved report column or if a duplicated result is found for the same metric, parameter configuration and XAI method.

`xai_metrics.reporting.reporting.save_attributions(context_outputs, output_dir)`

Save generated attributions to CSV files.

For each context output, the attribution matrix is converted into a dataframe using the stored observation identifiers as index and feature names as columns. Files are saved under `<output_dir>/<dataset>/<model>/<xai_method>/`.

Parameters

- **context_outputs** (Sequence[Mapping[str, Any]]) – Sequence of context output mappings. Each element must contain **metadata**, **attributions**, **observations** and **features**.
- **output_dir** (str or pathlib.Path) – Root directory where attribution files will be saved. Missing directories are created automatically.

Returns

Dictionary mapping each XAI method name to the path of the generated attribution CSV file.

Return type

Dict[str, str]

Raises

- **KeyError** – If any required field is missing from a context output.

`xai_metrics.reporting.reporting.save_reports(reports, output_dir, report_name='xai_metrics_report', observation_reports=None, experiment_started_at=None)`

Save summary reports and optional observation-level reports.

Summary report dataframes for every dataset and model are concatenated into a single report. Rows are sorted by dataset name, model name and metric name, while XAI method columns are sorted alphabetically.

The summary report is saved as both CSV and JSON. Structured CSV cell values, such as metric parameter dictionaries, are encoded as JSON strings. The JSON file contains the complete summary report as serialized records under the **"report"** key.

If **observation_reports** is provided, one CSV file is saved for each metric. These files preserve observation-level metric values and are stored in metric-specific subdirectories.

Parameters

- **reports** (Mapping[str, Mapping[str, pandas.DataFrame]]) – Nested dictionary of summary report dataframes, usually returned by `build_reports()`. The first level must be indexed by dataset name and the second level by model name.

- **output_dir** (*str* or *pathlib.Path*) – Directory where the report files will be saved. The directory and its missing parent directories are created automatically.
- **report_name** (*str*, *default*="xai_metrics_report") – Base filename used for the generated summary report files. A timestamp suffix is added automatically.
- **observation_reports** (*Mapping[str, Mapping[str, Mapping[str, pandas.DataFrame]]]* or *None*, *optional*) – Optional nested dictionary of observation-level reports, usually returned by [build_observation_reports\(\)](#).
- **experiment_started_at** (*Any* or *None*, *optional*) – Timestamp used to generate deterministic report filenames. If *None*, the current datetime is used.

Returns

Dictionary containing the generated summary report paths under "csv" and "json". If observation reports are saved, the dictionary also contains "observation_csv", mapping each metric name to its observation-level CSV path.

Return type

Dict[str, str]

Raises

ValueError – If `reports` does not contain any report dataframes to save.

EXPLAINERS

class xai_metrics.explainers.breakdown.**BreakDownExplainer**(context, params=None)

Bases: *BaseExplainer*

DALEX Break Down explainer.

This explainer computes local feature attributions using DALEX `predict_parts` with `type="break_down"`. For each observation, the DALEX contributions are converted into a fixed-size attribution vector whose order follows the columns of `context.X_background`.

The explainer supports classification and regression models through a configurable prediction function. For classification, the selected output column of `predict_proba` is used.

The explainer is registered under the name "BreakDown".

Parameters

- **context** (*ExplainerContext*) – Shared explainer context. It must contain `X_background` and may contain `y_background`. The background data columns define the order of the returned attribution vectors.
- **params** (*Mapping[str, Any] or None, optional*) – Explainer-specific parameters. Supported keys are:
 - `mode` : str, optional Model task type. If "classification", the explainer uses `predict_proba`. Otherwise, it uses `predict`. The default value is "classification".
 - `output_index` : int, optional Output column selected from `predict_proba` for classification models. The default value is 1.
 - `label` : str, optional Label passed to the DALEX explainer. If not provided, the model class name is used.
 - `verbose` : bool, optional Whether DALEX should print additional information. The default value is False.
 - `precalculate` : bool, optional Whether DALEX should precalculate model diagnostics. The default value is True.
 - `order` : Any, optional Feature order passed to `predict_parts`.
 - `n_samples` : int or None, optional Number of samples passed to `predict_parts` through `N`.
 - `keep_distributions` : bool, optional Whether DALEX should keep contribution distributions. The default value is False.
 - `random_state` : int, optional Random state passed to `predict_parts`. The default value is 42.

- **abs** : bool, optional Whether to return absolute attribution values. The default value is False.

If None, an empty dictionary is used.

explain(*model*, *inputs*, *targets=None*, ***kwargs*)

Generate Break Down attributions for the provided inputs.

The method converts the input data to a dataframe, obtains a cached DALEX explainer for the model and computes one **break_down** explanation per observation. The DALEX contributions are converted into attribution vectors ordered according to **context.X_background**.

Parameters

- **model** (*Any*) – Model whose predictions are to be explained.
- **inputs** (*Any*) – Input observation or batch of observations to explain.
- **targets** (*Any or None, optional*) – Unused by this explainer. The target output is controlled through the prediction function and, in classification mode, through **output_index**.
- ****kwargs** (*Any*) – Additional keyword arguments. They are accepted for compatibility with the common explainer interface.

Returns

Attribution matrix with one row per explained observation and one column per feature. If **abs=True**, absolute attribution values are returned.

Return type

numpy.ndarray

class xai_metrics.explainers.lime.LIMEExplainer(*context*, *params=None*)

Bases: [BaseExplainer](#)

LIME tabular explainer.

This explainer computes local feature attributions using `lime.lime_tabular.LimeTabularExplainer`. For each observation, LIME builds a local surrogate model around the input and returns feature weights for the selected target label.

The returned attributions are arranged as a fixed-size matrix whose columns follow the order of **context.X_background**. The explainer supports classification and regression models through **predict_proba** or **predict**, respectively.

The explainer is registered under the name "LIME".

Parameters

- **context** ([ExplainerContext](#)) – Shared explainer context. It must contain **X_background** and may contain **y_background**. The background data are used to initialise the LIME tabular explainer, and their columns define the order of the returned attribution vectors.
- **params** (*Mapping[str, Any] or None, optional*) – Explainer-specific parameters. Supported keys are:
 - **mode** : str, optional LIME mode. Usually "classification" or "regression". The default value is "classification".
 - **categorical_features** : list or None, optional Indices of categorical features passed to LIME.
 - **categorical_names** : dict or None, optional Mapping from categorical feature indices to category names.

- `kernel_width` : float or None, optional Width of the LIME kernel.
- `kernel` : callable or None, optional Custom kernel function used by LIME.
- `verbose` : bool, optional Whether LIME should print additional information. The default value is `False`.
- `class_names` : list or None, optional Class names used by LIME in classification mode.
- `feature_selection` : str, optional Feature selection strategy used by LIME. The default value is "auto".
- `discretize_continuous` : bool, optional Whether continuous features should be discretised. The default value is `True`.
- `discretizer` : str, optional Discretisation strategy used by LIME. The default value is "quartile".
- `sample_around_instance` : bool, optional Whether perturbations should be sampled around the explained instance. The default value is `False`.
- `random_state` : int, optional Random state used by LIME. The default value is 42.
- `labels` : tuple, list or None, optional Labels to explain. If not provided, the target labels passed to `explain()` are used. If no targets are provided, label 1 is used.
- `top_labels` : int or None, optional Number of top labels to explain. When provided, the first label returned by LIME is used to build the attribution vector.
- `num_features` : int, optional Maximum number of features included in each local explanation. The default value is the number of input features.
- `num_samples` : int, optional Number of perturbed samples generated by LIME. The default value is 500.
- `distance_metric` : str, optional Distance metric used by LIME. The default value is "euclidean".
- `model_regressor` : Any or None, optional Regressor used by LIME to fit the local surrogate model.

If None, an empty dictionary is used.

explain(*model*, *inputs*, *targets=None*, ***kwargs*)

Generate LIME attributions for the provided inputs.

The method converts the inputs to a NumPy array, resolves the prediction function and computes one LIME explanation per observation. The selected LIME weights are converted into attribution vectors ordered according to `context.X_background`.

If `labels` and `top_labels` are not configured, the method uses the provided `targets` as labels. If no targets are provided, label 1 is explained by default.

Parameters

- **model** (*Any*) – Model whose predictions are to be explained.
- **inputs** (*Any*) – Input observation or batch of observations to explain.
- **targets** (*Any or None, optional*) – Target labels to explain when `labels` and `top_labels` are not provided in `params`.
- ****kwargs** (*Any*) – Additional keyword arguments. They are accepted for compatibility with the common explainer interface.

Returns

Attribution matrix with one row per explained observation and one column per feature.

Return type

numpy.ndarray

class xai_metrics.explainers.maple.**MAPLEExplainer**(context, params=None)

Bases: *BaseExplainer*

MAPLE explainer.

This explainer computes local feature attributions using a MAPLE-style surrogate model. Background data are split into training and validation subsets. The training subset is used to fit the tree ensemble and local linear models, while the validation subset is used to select the retained feature subset.

For each observation, the explainer obtains the model response on the background data, builds or reuses a cached internal MAPLE model, and returns the coefficients of a local weighted Ridge model as attributions.

The explainer is based on MAPLE by Plumb, Molitor and Talwalkar (2018), “Model Agnostic Supervised Local Explanations”, and is registered under the name “MAPLE”.

Parameters

- **context** (*ExplainerContext*) – Shared explainer context. It must contain `X_background`. The background data columns define the order of the returned attribution vectors.
- **params** (*Mapping[str, Any] or None, optional*) – Explainer-specific parameters. Supported keys are:
 - `mode` : str, optional Model task type. If “classification”, the explainer uses `predict_proba`. Otherwise, it uses `predict`. The default value is “classification”.
 - `output_index` : int, optional Output column selected from two-dimensional model predictions. The default value is 1.
 - `validation_size` : float, optional Fraction of background observations used for validation. The default value is 0.2.
 - `data_type` : str, optional Type of background data. Supported values are “tabular” and “time_series”. For time series, the validation set is taken from the end of the background data. The default value is “tabular”.
 - `fe_type` : str, optional Tree ensemble used by MAPLE. Supported values are “rf” and “gbdt”. The default value is “rf”.
 - `n_estimators` : int, optional Number of trees in the ensemble. The default value is 200.
 - `max_features` : float, int, str or None, optional Maximum number of features considered by the ensemble. The default value is 0.5.
 - `min_samples_leaf` : int, optional Minimum number of samples required in each leaf. The default value is 10.
 - `regularization` : float, optional Ridge regularisation strength used by the local linear models. The default value is 0.001.
 - `random_state` : int or None, optional Random state used by the ensemble and background split. The default value is 42.
 - `abs` : bool, optional Whether to return absolute attribution values. The default value is False.

If None, an empty dictionary is used.

Raises

ValueError – If fewer than three background observations are available.

explain(*model*, *inputs*, *targets=None*, ***kwargs*)

Generate MAPLE attributions for the provided inputs.

The method converts the inputs to a NumPy array, obtains a cached internal MAPLE model and computes one local coefficient vector per observation. These coefficient vectors are returned as feature attributions.

Parameters

- **model** (*Any*) – Model whose predictions are to be explained.
- **inputs** (*Any*) – Input observation or batch of observations to explain.
- **targets** (*Any or None, optional*) – Unused by this explainer. The explained output is controlled by the model prediction function and, for two-dimensional outputs, by `output_index`.
- ****kwargs** (*Any*) – Additional keyword arguments. They are accepted for compatibility with the common explainer interface.

Returns

Attribution matrix with one row per explained observation and one column per feature. If `abs=True`, absolute attribution values are returned.

Return type

`numpy.ndarray`

class `xai_metrics.explainers.shap.SHAPExplainer`(*context*, *params=None*)

Bases: [`BaseExplainer`](#)

SHAP explainer.

This explainer computes feature attributions using `shap.Explainer`. Background data from `context.X_background` are used to initialise the SHAP explainer. For each batch of inputs, the SHAP values are converted into an attribution matrix whose columns follow the order of the background data.

For multi-output explanations, the output to explain can be selected through `output_index`, through the provided `targets` or through the context batch labels. If none of these are available, `default_output` is used.

The explainer is registered under the name "SHAP".

Parameters

- **context** ([`ExplainerContext`](#)) – Shared explainer context. It must contain `X_background` and may contain `y_batch`. The background data are used to initialise the SHAP explainer, and their columns define the order of the returned attribution vectors.
- **params** (*Mapping[str, Any] or None, optional*) – Explainer-specific parameters. Supported keys are:
 - `mode` : str, optional Model task type. If "classification", the explainer uses `predict_proba`. Otherwise, it uses `predict`. The default value is "classification".
 - `max_background_samples` : int or None, optional Maximum number of background observations used by SHAP. If provided, the background data are subsampled with `shap.sample()`.
 - `random_state` : int, optional Random state used for background sampling and SHAP explainer initialisation. The default value is 42.
 - `algorithm` : str, optional SHAP algorithm passed to `shap.Explainer`. The default value is "auto".

- `output_names` : list or None, optional Output names passed to `shap.Explainer`.
- `output_index` : int or None, optional Output index used when SHAP returns multi-output values. If provided, it takes precedence over `targets` and `context.y_batch`.
- `default_output` : int, optional Output index used when SHAP returns multi-output values and no explicit output index or targets are available. The default value is 1.
- `max_evals` : int or None, optional Maximum number of evaluations passed to SHAP. If None, "auto" is used.
- `batch_size` : int or None, optional Batch size passed to SHAP. If None, "auto" is used.
- `abs` : bool, optional Whether to return absolute SHAP values. The default value is False.

If None, an empty dictionary is used.

explain(*model*, *inputs*, *targets=None*, ***kwargs*)

Generate SHAP attributions for the provided inputs.

The method converts the inputs to a NumPy array, obtains a cached SHAP explainer and computes SHAP values for the batch. Multi-output values are reduced to a single output according to `output_index`, `targets`, `context.y_batch` or `default_output`.

Parameters

- **model** (*Any*) – Model whose predictions are to be explained.
- **inputs** (*Any*) – Input observation or batch of observations to explain.
- **targets** (*Any or None, optional*) – Target labels used to select the output dimension when SHAP returns multi-output values.
- ****kwargs** (*Any*) – Additional keyword arguments. They are accepted for compatibility with the common explainer interface.

Returns

Attribution matrix with one row per explained observation and one column per feature. If `abs=True`, absolute SHAP values are returned.

Return type

numpy.ndarray

METRICS

4.1 Complexity

`class xai_metrics.metrics.complexity.complexity_metric.Complexity(context, params=None)`

Bases: *BaseMetric*

Quantus Complexity metric.

This metric measures explanation complexity using the entropy of the attribution distribution. Quantus takes the absolute attribution values and expresses each feature contribution as a fraction of the total attribution magnitude.

Higher scores indicate that importance is distributed across more features and therefore corresponds to more complex explanations. Lower scores indicate that importance is concentrated on fewer features.

For an explanation with n features, the maximum entropy is approximately $\log(n)$ when importance is distributed uniformly. The scores returned by this wrapper are not divided by this maximum value.

The metric is based on the Complexity metric proposed by Bhatt et al. (2020) and implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - **normalise** : bool, optional Whether to normalise the attribution values before computing the metric. The default value is True.

If None, an empty dictionary is used.

Notes

The wrapper uses the default normalisation function provided by Quantus. Quantus also applies the absolute-value operation to the attributions because its `abs` parameter is not exposed by this wrapper and defaults to True.

run()

Compute the Complexity metric.

The method passes the selected input data, labels and attribution values to `quantus.Complexity`. Quantus flattens each explanation, takes its absolute values and computes the entropy of the fractional feature contributions.

If all attribution values are negative, this wrapper converts them to absolute values before calling Quantus. The model is set to training mode following the current wrapper implementation, although the metric itself depends only on the attribution values.

Returns

Complexity score for each evaluated observation. Lower values indicate that importance is concentrated on fewer features, while higher values indicate that it is more widely distributed. Scores are not normalised by the theoretical maximum $\log(n)$.

Return type

List[float]

class xai_metrics.metrics.complexity.sparseness.**Sparseness**(context, params=None)

Bases: *BaseMetric*

Quantus Sparseness metric.

This metric measures how concentrated the attribution magnitude is across the input features using the Gini index. Quantus applies the absolute-value operation to the attributions before computing the score.

Higher scores indicate that importance is concentrated on a smaller subset of features and therefore correspond to sparser explanations. Lower scores indicate a more uniform distribution of attribution magnitude.

The metric is based on the Sparseness metric proposed by Chalasani et al. (2020) and implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - **normalise** : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `True`.

If `None`, an empty dictionary is used.

Notes

The wrapper uses the default normalisation function provided by Quantus. Quantus applies the absolute-value operation to the attributions because its `abs` parameter is not exposed by this wrapper and defaults to `True`.

run()

Compute the Sparseness metric.

The method passes the selected input data, labels and attribution values to `quantus.Sparseness`. Quantus flattens each explanation, takes the absolute attribution values, sorts them and computes their Gini index.

If all attribution values are negative, this wrapper converts them to absolute values before calling Quantus. The model is set to training mode following the current wrapper implementation, although the metric itself depends only on the attribution values.

Returns

Sparseness score for each evaluated observation. Higher values indicate that attribution magnitude is concentrated on fewer features, while lower values indicate a more uniform distribution.

Return type

List[float]

4.2 Faithfulness

class xai_metrics.metrics.faithfulness.consistency.**Consistency**(context, params=None)

Bases: *BaseMetric*

Quantus Consistency metric.

This metric evaluates whether observations with equivalent explanations exhibit consistent model behaviour. Continuous attribution vectors are discretised by Quantus and observations sharing the same discrete representation are grouped together. For each observation, the metric computes the proportion of other observations in its group that receive the same predicted class.

Higher scores indicate stronger agreement between similar explanations and model predictions.

The metric is based on the Consistency metric proposed by Dasgupta et al. (2022) and implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - `abs` : bool, optional Whether to use absolute attribution values before computing the metric. If all attribution values are negative and this parameter is `True`, their absolute values are used. If it is `False`, the metric is skipped. The default value is `True`.
 - `normalise` : bool, optional Whether to normalise the attribution values before discretising the explanations. The default value is `True`.

If `None`, an empty dictionary is used.

Notes

The wrapper uses the default discretisation and normalisation functions provided by Quantus. The default discretisation function is `top_n_sign`.

run()

Compute the Consistency metric.

The method selects the observations defined in the metric context and passes their input data, labels and attribution values to `quantus.Consistency`. Quantus discretises the explanations, predicts the class of each observation and compares predictions among observations with equivalent discrete explanations.

If all attribution values are negative, their treatment depends on the `abs` parameter. Their absolute values are used when `abs=True`; otherwise, the metric is skipped.

The model is set to evaluation mode before the metric is computed.

Returns

Consistency score for each evaluated observation. Higher values indicate that observations with the same discretised explanation more frequently receive the same predicted class.

Return type

List[float]

Raises

MetricSkipped – If all attribution values are negative and `abs` is `False`.

```
class xai_metrics.metrics.faithfulness.faithfulness.Faithfulness(context, params=None)
```

Bases: *BaseMetric*

AIX360 Faithfulness metric.

This metric evaluates whether feature attribution values reflect the influence of the corresponding features on the model prediction. For each observation, every feature is individually replaced with a baseline value and the probability assigned to the original predicted class is recorded. The score is the negative Pearson correlation between the attribution values and these probabilities.

Higher scores indicate stronger agreement between feature importance and the effect of replacing features with their baseline values.

The wrapped AIX360 implementation requires a classification model exposing a `predict_proba` method.

The metric is based on the faithfulness criterion proposed by Alvarez-Melis and Jaakkola (2018) and implemented in AIX360.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values. The model must implement `predict_proba`.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - `base_values` : array-like or None, optional Explicit baseline values used to replace feature values. If provided, this takes priority over `base_func` and `base_strategy`.
 - `base_strategy` : str, optional Strategy used to compute baseline values from `X_test` when `base_values` and `base_func` are not provided. Supported values are "mean", "median" and "zero". The default value is "mean".

If None, an empty dictionary is used.

`run()`

Compute the Faithfulness metric.

The method resolves a common baseline vector from the complete test dataset and evaluates each selected observation independently using `aix360.metrics.faithfulness_metric()`.

For each observation, AIX360 determines the predicted class, replaces each feature individually with its baseline value and records the resulting probability for that class. The returned score is the negative Pearson correlation between these probabilities and the attribution values.

Returns

Faithfulness score for each evaluated observation. Higher values indicate that features with larger attribution values produce greater decreases in the predicted class probability when replaced.

Return type

List[float]

Raises

- **ValueError** – If `base_strategy` is not "mean", "median" or "zero".
- **AttributeError** – If the model does not implement the `predict_proba` method required by AIX360.

Notes

The score may be nan when the attribution values or the probabilities obtained after feature replacement have zero variance.

```
class xai_metrics.metrics.faithfulness.faithfulness_estimate.FaithfulnessEstimate(context,
                                                                              params=None)
```

Bases: *BaseMetric*

Quantus Faithfulness Estimate metric.

This metric evaluates whether features with higher attribution values produce larger changes in the model output when they are perturbed. Features are perturbed in groups ordered by attribution importance, and the metric compares the sum of their attributions with the corresponding drop in the target output.

This wrapper uses a safe Pearson correlation function that returns 0.0 when either of the compared vectors has zero variance, avoiding undefined correlation values.

Higher scores indicate stronger agreement between attribution importance and changes in the model output.

The metric is based on the Faithfulness Estimate proposed by Alvarez-Melis and Jaakkola (2018) and implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - `features_in_step` : int, optional Number of features perturbed at each step. The default value is 1.
 - `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `False`.
 - `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `True`.
 - `perturb_baseline` : str, optional Baseline value used when perturbing features. Supported values depend on the Quantus perturbation function. Common values are "black", "white", "mean", "random" and "uniform". The default value is "black".

If `None`, an empty dictionary is used.

Notes

This wrapper uses its internal `_safe_pearson()` method as the similarity function. The default normalisation and perturbation functions provided by Quantus are used.

run()

Compute the Faithfulness Estimate metric.

The method selects the observations defined in the metric context and passes their input data, labels and attribution values to `quantus.FaithfulnessEstimate`. Features are perturbed in groups, and the internal safe Pearson correlation function compares attribution sums with the resulting target-output drops.

If all attribution values are negative, their treatment depends on the `abs` parameter. Their absolute values are used when `abs=True`; otherwise, the metric is skipped.

The model is set to evaluation mode before the metric is computed.

Returns

Faithfulness Estimate score for each evaluated observation. Higher values indicate stronger agreement between attribution importance and model output changes after perturbation.

Return type

List[float]

Raises

MetricSkipped – If all attribution values are negative and `abs` is `False`.

class xai_metrics.metrics.faithfulness.monotonicity.**Monotonicity**(context, params=None)

Bases: *BaseMetric*

Quantus Monotonicity metric.

This metric evaluates whether the target model output increases monotonically as features are progressively introduced from a baseline input. Features are processed in increasing order of attribution value and may be introduced individually or in groups.

For each observation, the metric returns `True` when the sequence of target outputs is monotonically non-decreasing and `False` otherwise.

The metric is based on the Monotonicity metric proposed by Arya et al. (2019) and the monotonic attribute functions described by Luss et al. (2019), as implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - `features_in_step` : int, optional Number of features added at each perturbation step. The default value is 1.
 - `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `True`.
 - `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `True`.
 - `perturb_baseline` : str, optional Baseline value used to initialise the perturbed input. Supported values depend on the Quantus perturbation function. Common values are "black", "white", "mean", "random" and "uniform". The default value is "black".

If `None`, an empty dictionary is used.

Notes

The wrapper uses the default normalisation and perturbation functions provided by Quantus.

run()

Compute the Monotonicity metric.

The method passes the selected input data, target labels and attribution values to `quantus.Monotonicity`. For each observation, Quantus starts from a baseline input, processes features in increasing order of attribution value and evaluates the target model output after each group is introduced.

If all attribution values are negative, their absolute values are used when `abs=True`; otherwise, the metric is skipped. The model is set to evaluation mode before the computation.

Returns

Monotonicity result for each evaluated observation. True indicates that the target output is monotonically non-decreasing across the feature-introduction steps.

Return type

List[bool]

Raises

MetricSkipped – If all attribution values are negative and abs is False.

```
class xai_metrics.metrics.faithfulness.monotonicity_correlation.MonotonicityCorrelation(context,
                                                                                      params=None)
```

Bases: *BaseMetric*

Quantus Monotonicity Correlation metric.

This metric evaluates whether feature attribution values are monotonically related to the uncertainty caused by perturbing the corresponding features. Features are grouped in increasing order of attribution, and each group is perturbed repeatedly to estimate its effect on the target model output. The score is the Spearman correlation between the attribution sums and the estimated output variations.

This wrapper uses a safe Spearman correlation that returns 0.0 when either input vector has zero variance.

Higher scores indicate a stronger positive relationship between feature importance and the uncertainty caused by perturbing those features.

The metric is based on the Monotonicity Correlation metric proposed by Nguyen and Rodríguez Martínez (2020) and implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, X_test, y_test, selected observations and attribution values.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - **eps** : float, optional Threshold used when computing the inverse prediction factor. The default value is 1e-5.
 - **nr_samples** : int, optional Number of perturbation samples generated for each feature group. The default value is 100.
 - **features_in_step** : int, optional Number of features perturbed at each step. The default value is 1.
 - **abs** : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is True.
 - **normalise** : bool, optional Whether to normalise the attribution values before computing the metric. The default value is True.
 - **perturb_baseline** : str, optional Baseline value used when perturbing features. Supported values depend on the Quantus perturbation function. Common values are "black", "white", "mean", "random" and "uniform". The default value is "uniform".

If None, an empty dictionary is used.

Notes

The wrapper uses `_safe_spearman()` as the similarity function and the default normalisation and perturbation functions provided by Quantus.

`run()`

Compute the Monotonicity Correlation metric.

The method passes the selected inputs, labels and attribution values to `quantus.MonotonicityCorrelation`. Quantus perturbs groups of features repeatedly, estimates their relative effect on the target output and compares those estimates with the corresponding attribution sums using `_safe_spearman()`.

If all attribution values are negative, their absolute values are used when `abs=True`; otherwise, the metric is skipped. The model is set to evaluation mode before the computation.

Returns

Monotonicity Correlation score for each evaluated observation. Higher values indicate a stronger positive relationship between attribution importance and output variation.

Return type

List[float]

Raises

MetricSkipped – If all attribution values are negative and `abs` is `False`.

```
class xai_metrics.metrics.faithfulness.monotonicity_metric.MonotonicityMetric(context,  
                                                                           params=None)
```

Bases: *BaseMetric*

AIX360 Monotonicity metric.

This metric evaluates whether the probability assigned to the originally predicted class increases monotonically as features are progressively restored from a baseline input.

For each observation, features are restored in increasing order of their attribution values. The metric returns `True` when the resulting sequence of predicted-class probabilities is monotonically non-decreasing.

The wrapped AIX360 implementation requires a classification model exposing a `predict_proba` method.

The metric is based on the monotonicity criterion described by Luss et al. (2019) and implemented in AIX360.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values. The model must implement `predict_proba`.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - `base_values` : array-like or `None`, optional Explicit baseline values used as the initial feature values. If provided, this takes priority over `base_func` and `base_strategy`.
 - `base_strategy` : str, optional Strategy used to compute baseline values from `X_test` when `base_values` and `base_func` are not provided. Supported values are "mean", "median" and "zero". The default value is "mean".

If `None`, an empty dictionary is used.

`run()`

Compute the Monotonicity metric.

The method resolves a common baseline vector from the complete test dataset and evaluates each selected observation independently using `aix360.metrics.monotonicity_metric()`.

For each observation, AIX360 determines the class predicted for the original input. Starting from the baseline, it restores features cumulatively in increasing order of attribution value and evaluates the probability assigned to that class after each restoration.

Returns

Monotonicity result for each evaluated observation. `True` indicates that the predicted-class probability never decreases as features are progressively restored.

Return type

`List[bool]`

Raises

- **ValueError** – If `base_strategy` is not "mean", "median" or "zero".
- **AttributeError** – If the model does not implement the `predict_proba` method required by AIX360.

class `xai_metrics.metrics.fairness.sensitivity_n.SensitivityN`(*context*, *params*=None)

Bases: `BaseMetric`

Quantus Sensitivity-N metric.

This metric evaluates the agreement between feature attribution values and the variation in the target model output caused by perturbing the corresponding features.

Quantus orders features by decreasing attribution value and progressively perturbs them in groups. At each step, it compares the target-output change with the attribution sum of the processed feature group using Pearson correlation across the evaluated observations.

The number of perturbation steps is limited by `n_max_percentage`. Higher correlation values indicate stronger agreement between the explanation and the model behaviour.

The metric is based on the Sensitivity-N test proposed by Ancona et al. (2018) and implemented in Quantus.

Parameters

- **context** (`MetricContext`) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.
- **params** (`Mapping[str, Any]` or `None`, optional) – Metric-specific parameters. Supported keys are:
 - `n_max_percentage` : float, optional Maximum percentage of features to evaluate. The default value is `0.8`.
 - `features_in_step` : int, optional Number of features perturbed at each step. The default value is `1`.
 - `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `False`.
 - `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `True`.
 - `perturb_baseline` : str, optional Baseline value used when perturbing features. Supported values depend on the Quantus perturbation function. Common values are "black", "white", "mean", "random" and "uniform". The default value is "black".

If `None`, an empty dictionary is used.

Notes

The wrapper uses the default Pearson correlation, normalisation and baseline-replacement functions provided by Quantus. Quantus aggregates the step-wise correlations by default.

run()

Compute the Sensitivity-N metric.

The method passes the selected input data, target labels and attribution values to `quantus.SensitivityN`. Quantus progressively perturbs feature groups ordered by decreasing attribution value and computes the Pearson correlation between their attribution sums and the resulting target-output changes.

Only the perturbation steps covered by `n_max_percentage` are evaluated. If all attribution values are negative, their absolute values are used when `abs=True`; otherwise, the metric is skipped. The model is set to evaluation mode before the computation.

Returns

Sensitivity-N result returned by Quantus. Higher values indicate stronger agreement between attribution values and target-output changes. With the default Quantus configuration, the step-wise correlations are aggregated.

Return type

List[float]

Raises

MetricSkipped – If all attribution values are negative and `abs` is `False`.

class `xai_metrics.metrics.fairness.sufficiency.Sufficiency(context, params=None)`

Bases: *BaseMetric*

Quantus Sufficiency metric.

This metric evaluates whether observations with similar explanations receive the same model prediction. Quantus computes the pairwise distances between attribution vectors and considers two explanations similar when their distance is less than or equal to `threshold`.

For each observation, the score is the proportion of other observations with similar explanations that receive the same predicted class. A score of `0.0` is returned when no other explanation satisfies the distance threshold.

Higher scores indicate stronger agreement between similar explanations and model predictions. Since explanations are compared within the evaluated batch, the results depend on the selected observations.

The metric is based on the Sufficiency test proposed by Dasgupta et al. (2022) and implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - **threshold** : float, optional Maximum distance between two attribution vectors for their explanations to be considered similar. The default value is `0.6`.
 - **distance_func** : str, optional Distance function used to compare attribution vectors. The value is passed to Quantus and is typically a valid SciPy distance name, such as "seuclidean". The default value is "seuclidean".
 - **abs** : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `True`.

- `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `True`.

If `None`, an empty dictionary is used.

Notes

The wrapper uses the default normalisation function provided by Quantus. Scores depend on the observations evaluated in the same call, since these observations define the explanation neighbourhoods.

`run()`

Compute the Sufficiency metric.

The method passes the selected input data, labels and attribution values to `quantus.Sufficiency`. Quantus compares the attribution vectors, identifies observations with similar explanations and computes the proportion that receive the same predicted class. Self-comparisons are excluded.

If all attribution values are negative, their absolute values are used when `abs=True`; otherwise, the metric is skipped. The model is set to evaluation mode before the computation.

Returns

Sufficiency score for each evaluated observation. Scores range from `0.0` to `1.0`. Higher values indicate that observations with similar explanations more frequently receive the same predicted class.

Return type

List[float]

Raises

`MetricSkipped` – If all attribution values are negative and `abs` is `False`.

4.3 Fidelity

4.3.1 Completeness

`class xai_metrics.metrics.fidelity.completeness.average_drop.AverageDrop(context, params=None)`

Bases: *BaseMetric*

Average Drop fidelity metric.

This metric evaluates how much the model score decreases when the original input is masked according to the importance values provided by the explanation. Features with larger absolute attribution values are preserved to a greater extent, while features with smaller attribution values are attenuated.

For each observation i , the metric is computed as:

$$\begin{aligned} \text{base}_i &= g(f, \mathbf{x}_i, y_i), \\ \text{after}_i &= g(f, \mathbf{x}_i \odot M_i, y_i), \\ \text{AD}_i &= \frac{\max(\text{base}_i - \text{after}_i, 0)}{\text{base}_i + \varepsilon} \end{aligned}$$

where f is the model, g is the scoring operator and M_i is a normalised mask derived from the attribution values.

Attribution values are first converted to absolute values and independently min-max normalised to the interval $[0, 1]$ for each observation. The resulting mask is then broadcast to the input shape when necessary and multiplied element-wise by the original input.

The metric returns one Average Drop value per observation. Lower values are better: a small score indicates that retaining the features considered important by the explanation preserves the model score. A value of zero indicates that masking the input does not decrease the score, or that it increases it.

The implementation is based on the Average Drop metric introduced in Chattopadhyay et al. (2018) and follows the implementation provided by Xplique, with support for PyTorch models, models exposing `predict` or `predict_proba`, and custom scoring operators.

Chattopadhyay, A., Sarkar, A., Howlader, P., & Balasubramanian, V. N. (2018). Grad-CAM++: Generalized Gradient-Based Visual Explanations for Deep Convolutional Networks. In 2018 IEEE Winter Conference on Applications of Computer Vision (WACV), pp. 839-847.

Parameters

- **context** (`MetricContext`) – Shared metric evaluation context. It must contain the model, `X_test`, selected observations and attribution values. `y_test` may be `None` when the model score can be determined without explicit targets.
- **params** (`Mapping[str, Any]` or `None`, optional) – Metric-specific parameters. Supported keys are:
 - `batch_size` : int or `None`, optional Maximum number of observations processed at once. If `None`, all selected observations are processed together. The default value is 64.
 - `operator` : callable or `None`, optional Custom scoring function `g(model, inputs, targets)` used to obtain one scalar score per observation. If `None`, the metric performs inference directly using the model and selects the corresponding target score. The default value is `None`.
 - `activation` : {`None`, “sigmoid”, “softmax”}, optional Activation function applied to model predictions before selecting the score. `None` leaves predictions unchanged. The default value is `None`.

If `None`, an empty dictionary is used.

Notes

When no custom `operator` is provided, model inference is performed using, in order, a PyTorch `torch.nn.Module`, `predict_proba`, `predict` or direct model invocation.

For two-dimensional model outputs, targets are interpreted as class indices. If no targets are provided, the maximum model output for each observation is used.

For image explanations with shape (N, H, W, C), attribution values are averaged over the last dimension before constructing the mask. For time-series explanations with shape (N, T), the resulting mask is broadcast over the feature dimension of inputs shaped (N, T, F).

run()

Compute the Average Drop metric.

The method selects the observations defined in the metric context, computes their original model scores and creates perturbed inputs by multiplying each observation by a normalised attribution mask. Model scores are then recomputed using the perturbed inputs.

For each observation, Average Drop is computed as:

$$AD_i = \frac{\max(\text{base}_i - \text{after}_i, 0)}{\text{base}_i + \varepsilon}$$

where base_i is the model score for the original input and after_i is the score obtained after retaining the input according to the explanation mask.

The ReLU operation in the numerator prevents increases in the model score after masking from producing negative Average Drop values.

Returns

Average Drop score for each evaluated observation. Lower values are better. A value of 0 indicates that the explanation-based mask preserves or increases the original model score.

Return type

List[float]

Raises

- ***MetricSkipped*** – If no observations are selected.
- **ValueError** – If `batch_size` is not positive, if `activation` is not one of `None`, `"sigmoid"` or `"softmax"`, if the number of explanations differs from the number of selected observations, or if explanation and input shapes are incompatible.

Notes

The interpretation of the metric assumes that the selected model score is positive. This is typically the case when probabilities or other non-negative confidence scores are used. When raw logits or arbitrary custom operator outputs are used, the usual interpretation and expected range of Average Drop may not hold.

```
class xai_metrics.metrics.fidelity.completeness.average_gain.AverageGain(context,
                                                                    params=None)
```

Bases: *BaseMetric*

Average Gain fidelity metric.

This metric evaluates how much the model score increases when the original input is masked according to the importance values provided by the explanation. Features with larger absolute attribution values are preserved to a greater extent, while features with smaller attribution values are attenuated.

For each observation i , the metric is computed as:

$$\begin{aligned} \text{base}_i &= g(f, \mathbf{x}_i, y_i), \\ \text{after}_i &= g(f, \mathbf{x}_i \odot M_i, y_i), \\ \text{AG}_i &= \frac{\max(\text{after}_i - \text{base}_i, 0)}{1 - \text{base}_i + \varepsilon} \end{aligned}$$

where f is the model, g is the scoring operator and M_i is a normalised mask derived from the attribution values.

Attribution values are first converted to absolute values and independently min-max normalised to the interval $[0, 1]$ for each observation. The resulting mask is then broadcast to the input shape when necessary and multiplied element-wise by the original input.

The metric returns one Average Gain value per observation. Higher values are better: a large score indicates that retaining the features considered important by the explanation increases the model score relative to the original input. A value of zero indicates that masking the input does not increase the score.

Average Gain is complementary to Average Drop. While Average Drop measures the relative decrease in the model score after explanation-based masking, Average Gain measures the relative increase with respect to the remaining score range up to one.

The metric is intended for model scores in the interval $[0, 1]$. If the model produces logits, `activation="softmax"` or `activation="sigmoid"` should be used, or a custom operator returning probability-like scores should be provided.

The implementation is based on the Average Gain metric described by Zhang et al. (2024) and follows the implementation provided by Xplique, with support for PyTorch models, models exposing `predict` or `predict_proba`, and custom scoring operators.

Zhang, H., Torres, F., Sicre, R., Avrithis, Y., & Ayache, S. (2024). Opti-CAM: Optimizing Saliency Maps for Interpretability. *Computer Vision and Image Understanding*, 248, 104101.

Parameters

- **context** (`MetricContext`) – Shared metric evaluation context. It must contain the model, `X_test`, selected observations and attribution values. `y_test` may be `None` when the model score can be determined without explicit targets.
- **params** (`Mapping[str, Any]` or `None`, optional) – Metric-specific parameters. Supported keys are:
 - `batch_size` : int or `None`, optional Maximum number of observations processed at once. If `None`, all selected observations are processed together. The default value is 64.
 - `operator` : callable or `None`, optional Custom scoring function `g(model, inputs, targets)` used to obtain one scalar score per observation. The function should return scores in the interval `[0, 1]`. If `None`, the metric performs inference directly using the model and selects the corresponding target score. The default value is `None`.
 - `activation` : `{None, "sigmoid", "softmax"}`, optional Activation function applied to model predictions before selecting the score. `None` leaves predictions unchanged. Since Average Gain requires scores in `[0, 1]`, "sigmoid" or "softmax" should be used when the model returns logits. The default value is `None`.

If `None`, an empty dictionary is used.

Notes

When no custom `operator` is provided, model inference is performed using, in order, a PyTorch `torch.nn.Module`, `predict_proba`, `predict` or direct model invocation.

For two-dimensional model outputs, targets are interpreted as class indices. If no targets are provided, the maximum model output for each observation is used.

For image explanations with shape `(N, H, W, C)`, attribution values are averaged over the last dimension before constructing the mask. For time-series explanations with shape `(N, T)`, the resulting mask is broadcast over the feature dimension of inputs shaped `(N, T, F)`.

`run()`

Compute the Average Gain metric.

The method selects the observations defined in the metric context, computes their original model scores and creates perturbed inputs by multiplying each observation by a normalised attribution mask. Model scores are then recomputed using the perturbed inputs.

For each observation, Average Gain is computed as:

$$AG_i = \frac{\max(\text{after}_i - \text{base}_i, 0)}{1 - \text{base}_i + \varepsilon}$$

where `basei` is the model score for the original input and `afteri` is the score obtained after retaining the input according to the explanation mask.

The numerator measures the positive increase in model score produced by explanation-based masking. The denominator normalises this increase by the remaining possible increase from the original score to one.

The ReLU operation in the numerator prevents decreases in the model score after masking from producing negative Average Gain values.

Returns

Average Gain score for each evaluated observation. Higher values are better. A value of 0 indicates that the explanation-based mask does not increase the original model score.

Return type

List[float]

Raises

- ***MetricSkipped*** – If no observations are selected, or if any original model score lies outside the interval [0, 1].
- ***ValueError*** – If `batch_size` is not positive, if `activation` is not one of `None`, `"sigmoid"` or `"softmax"`, if the number of explanations differs from the number of selected observations, or if explanation and input shapes are incompatible.

Notes

Average Gain is designed for model scores in the interval [0, 1]. When the model returns logits, `activation="softmax"` or `activation="sigmoid"` should be used. Alternatively, a custom operator can be provided as long as it returns probability-like scores.

Unlike Average Drop, which normalises a decrease by the original model score, Average Gain normalises an increase by $1 - \text{base_score}$. This represents the remaining score range available above the original prediction.

```
class xai_metrics.metrics.fidelity.completeness.average_increase.AverageIncrease(context,
                                                                              params=None)
```

Bases: *BaseMetric*

Average Increase in Confidence fidelity metric.

This metric evaluates whether the model score increases when the original input is masked according to the importance values provided by the explanation. Features with larger absolute attribution values are preserved to a greater extent, while features with smaller attribution values are attenuated.

For each observation i , the metric is computed as:

$$\begin{aligned}\text{base}_i &= g(f, \mathbf{x}_i, y_i), \\ \text{after}_i &= g(f, \mathbf{x}_i \odot M_i, y_i), \\ \text{AIC}_i &= \mathbb{I}[\text{after}_i > \text{base}_i]\end{aligned}$$

where f is the model, g is the scoring operator, M_i is a normalised mask derived from the attribution values and $\mathbb{I}[\cdot]$ denotes the indicator function.

Attribution values are first converted to absolute values and independently min-max normalised to the interval [0, 1] for each observation. The resulting mask is then broadcast to the input shape when necessary and multiplied element-wise by the original input.

The metric returns one binary value per observation. A value of 1 indicates that retaining the features considered important by the explanation increases the model score, while a value of 0 indicates that the score remains unchanged or decreases.

Higher values are better when the results are aggregated, since the mean of the binary indicators represents the proportion of observations for which explanation-based masking increases the model score.

Average Increase is closely related to Average Drop and Average Gain, but only considers whether an increase occurs, independently of its magnitude.

The metric is generally most meaningful when applied to confidence or probability-like model scores. If the model produces logits, `activation="softmax"` or `activation="sigmoid"` can be used before computing the comparison.

The implementation is based on the Average Increase in Confidence metric introduced by Chattopadhyay et al. (2018) and follows the implementation provided by Xplique, with support for PyTorch models, models exposing `predict` or `predict_proba`, and custom scoring operators.

Chattopadhyay, A., Sarkar, A., Howlader, P., & Balasubramanian, V. N. (2018). Grad-CAM++: Generalized Gradient-Based Visual Explanations for Deep Convolutional Networks. In 2018 IEEE Winter Conference on Applications of Computer Vision (WACV), pp. 839-847.

Parameters

- **context** (`MetricContext`) – Shared metric evaluation context. It must contain the model, `X_test`, selected observations and attribution values. `y_test` may be `None` when the model score can be determined without explicit targets.
- **params** (`Mapping[str, Any]` or `None`, optional) – Metric-specific parameters. Supported keys are:
 - `batch_size` : int or `None`, optional Maximum number of observations processed at once. If `None`, all selected observations are processed together. The default value is 64.
 - `operator` : callable or `None`, optional Custom scoring function `g(model, inputs, targets)` used to obtain one scalar score per observation. If `None`, the metric performs inference directly using the model and selects the corresponding target score. The default value is `None`.
 - `activation` : {`None`, “sigmoid”, “softmax”}, optional Activation function applied to model predictions before selecting the score. `None` leaves predictions unchanged. Probability-like scores are generally preferred for interpreting Average Increase. The default value is `None`.

If `None`, an empty dictionary is used.

Notes

When no custom `operator` is provided, model inference is performed using, in order, a PyTorch `torch.nn.Module`, `predict_proba`, `predict` or direct model invocation.

For two-dimensional model outputs, targets are interpreted as class indices. If no targets are provided, the maximum model output for each observation is used.

For image explanations with shape (N, H, W, C), attribution values are averaged over the last dimension before constructing the mask. For time-series explanations with shape (N, T), the resulting mask is broadcast over the feature dimension of inputs shaped (N, T, F).

run()

Compute the Average Increase metric.

The method selects the observations defined in the metric context, computes their original model scores and creates perturbed inputs by multiplying each observation by a normalised attribution mask. Model scores are then recomputed using the perturbed inputs.

For each observation, Average Increase is computed as:

$$AIC_i = \mathbb{I}[\text{after}_i > \text{base}_i]$$

where `basei` is the model score for the original input, `afteri` is the score obtained after applying the explanation-based mask.

Consequently, each observation receives a score of 1 when explanation-based masking increases the model score and 0 otherwise.

Returns

Binary Average Increase indicator for each evaluated observation. Values are 1.0 when the explanation-based mask increases the model score and 0.0 when the score remains unchanged or decreases. Higher values are better.

Return type

List[float]

Raises

- ***MetricSkipped*** – If no observations are selected.
- **ValueError** – If `batch_size` is not positive, if `activation` is not one of `None`, `"sigmoid"` or `"softmax"`, if the number of explanations differs from the number of selected observations, or if explanation and input shapes are incompatible.

Notes

The returned values are per-observation binary indicators. When aggregated using their mean, the result represents the proportion of evaluated observations for which retaining the features identified as important by the explanation increases the model score.

Although the comparison can be applied directly to arbitrary scalar model outputs, Average Increase is generally interpreted in terms of model confidence. Probability-like outputs are therefore recommended. When the model returns logits, `activation="softmax"` or `activation="sigmoid"` can be used before computing the metric.

```
class xai_metrics.metrics.fidelity.completeness.completeness_metric.Completeness(context,
                                                                                    params=None)
```

Bases: *BaseMetric*

Quantus Completeness metric.

This metric evaluates whether the sum of the attribution values matches the difference between the model output for the original input and the model output for a baseline input. This property is also known as Summation to Delta or Conservation.

The metric returns one boolean value per observation. A value of `True` indicates that the attribution sum satisfies the completeness condition for the selected model output.

Higher scores are better when the boolean results are aggregated, since a larger proportion of `True` values indicates stronger agreement with the completeness axiom.

The metric is based on the Completeness property proposed by Sundararajan et al. (2017), the Summation to Delta property proposed by Shrikumar et al. (2017), and the Conservation property discussed by Montavon et al. (2018), as implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `False`.
 - `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `True`.
 - `perturb_baseline` : str, optional Baseline value used to create the reference input. Supported values depend on the Quantus perturbation function. Common values are `"black"`, `"white"`, `"mean"`, `"random"` and `"uniform"`. The default value is `"black"`.

If `None`, an empty dictionary is used.

Notes

The wrapper uses the default functions provided by Quantus. The default perturbation function replaces all features by the configured baseline, and the default output transformation is the identity function.

Quantus computes this metric using logits by default, not softmax probabilities.

run()

Compute the Completeness metric.

The method selects the observations defined in the metric context and passes their input data, labels and attribution values to `quantus.Completeness`. Quantus replaces the input features by a baseline, computes the difference between the model output at the original input and at the baseline input, and compares this difference with the sum of the attribution values.

If all attribution values are negative, their treatment depends on the `abs` parameter. Their absolute values are used when `abs=True`; otherwise, the metric is skipped.

The model is set to evaluation mode before the metric is computed.

Returns

Completeness result for each evaluated observation. `True` indicates that the sum of the attribution values matches the output difference between the original input and the baseline input.

Return type

List[bool]

Raises

MetricSkipped – If all attribution values are negative and `abs` is `False`.

class `xai_metrics.metrics.fidelity.completeness.deletion.Deletion`(*context*, *params=None*)

Bases: *BaseMetric*

Deletion fidelity metric.

This metric evaluates whether the features identified as important by an explanation are relevant to the model prediction. Starting from the original input, features are progressively replaced by a baseline value following their attribution-based importance ranking.

For each observation, features with larger absolute attribution values are removed first. After each deletion step, the model score is recomputed, producing a deletion curve that describes how the score evolves as increasingly important features are replaced by the baseline.

A faithful explanation is expected to identify features that strongly influence the model output. Consequently, removing highly important features should produce a rapid decrease in the model score and therefore a small area under the deletion curve. Lower values are better.

For each observation i , the procedure can be described as:

$$\begin{aligned} \mathbf{x}_i^{(0)} &= \mathbf{x}_i, \\ \mathbf{x}_i^{(k)} &= \text{delete}(\mathbf{x}_i, M_i, k), \\ s_i^{(k)} &= g(f, \mathbf{x}_i^{(k)}, y_i) \end{aligned}$$

where f is the model, g is the scoring operator, M_i defines the feature ranking induced by the explanation and k is the number of perturbed features.

If the deletion curve contains $T + 1$ point, the reported score is its normalized trapezoidal area:

$$\text{Deletion}_i = \frac{1}{T} \sum_{t=0}^{T-1} \frac{s_i^{(t)} + s_i^{(t+1)}}{2}$$

A lower value indicates that deleting features considered important by the explanation causes the model score to decrease more rapidly.

The implementation is based on the Deletion metric introduced by Petsiuk et al. (2018) and follows the implementation provided by Xplique, with support for PyTorch models, models exposing `predict` or `predict_proba`, custom scoring operators and configurable baseline values.

Petsiuk, V., Das, A., & Saenko, K. (2018). RISE: Randomized Input Sampling for Explanation of Black-box Models. In British Machine Vision Conference (BMVC).

Parameters

- **context** (`MetricContext`) – Shared metric evaluation context. It must contain the model, `X_test`, selected observations and attribution values. `y_test` may be `None` when the model score can be determined without explicit targets.
- **params** (`Mapping[str, Any]` or `None`, optional) – Metric-specific parameters. Supported keys are:
 - `batch_size` : int, optional Maximum number of perturbed observations processed at once when computing model scores. The default value is 64.
 - `baseline_mode` : float, array-like or callable, optional Baseline used to replace deleted features. A scalar value is broadcast to the input shape. If callable, it receives the selected input observations and must return a value or array broadcastable to their shape. The default value is `0.0`.
 - `steps` : int, optional Number of deletion intervals between the original input and the maximum perturbation level. A value of -1 evaluates every possible number of deleted features up to the configured maximum. The default value is 10.
 - `max_percentage_perturbed` : float, optional Maximum fraction of input features that may be replaced by the baseline. Must belong to the interval `(0, 1]`. The default value is 1.0.
 - `operator` : callable or `None`, optional Custom scoring function `g(model, inputs, targets)` used to obtain one scalar score per observation. If `None`, the metric performs inference directly using the model and selects the corresponding target score. The default value is `None`.
 - `activation` : {`None`, “sigmoid”, “softmax”}, optional Activation function applied to model predictions before selecting the score. `None` leaves predictions unchanged. The default value is `None`.

If `None`, an empty dictionary is used.

Notes

When no custom `operator` is provided, model inference is performed using, in order, a PyTorch `torch.nn.Module`, `predict_proba`, `predict` or direct model invocation.

For two-dimensional model outputs, targets are interpreted as class indices. If no targets are provided, the maximum model output for each observation is used.

Inputs and explanations are flattened before the deletion process. Therefore, each explanation must contain exactly the same number of feature values as its corresponding input observation.

Features are ranked directly by their attribution values in descending order. The metric therefore removes the largest attribution values first.

run()

Compute the Deletion metric.

The method selects the observations defined in the metric context and progressively replaces their features by a configured baseline. Features are deleted in descending order of their attribution values, so that the features considered most important by the explanation are perturbed first.

Inputs and explanations are flattened before ranking and perturbation. For each deletion step, the selected number of features is replaced by the corresponding baseline values and the model score is recomputed.

The sequence of scores forms a deletion curve for each observation. The final score is the normalized trapezoidal area of that curve:

$$\text{Deletion}_i = \frac{1}{T} \sum_{t=0}^{T-1} \frac{s_i^{(t)} + s_i(t+1)}{2}$$

where $s_i^{(t)}$ is the model score after deleting t features and T is the number of curve intervals.

Lower scores are better. A small value indicates that removing features identified as important by the explanation rapidly decreases the model score, suggesting that the explanation successfully captures features relevant to the prediction.

Returns

Deletion score for each evaluated observation. Lower values indicate better explanations.

Return type

List[float]

Raises

- ***MetricSkipped*** – If no observations are selected.
- ***ValueError*** – If the number of explanations differs from the number of selected observations, if inputs and explanations do not contain the same number of features per observation, if `batch_size` is not positive, if `max_percentage_perturbed` does not belong to `(0, 1]`, if `steps` is neither positive nor `-1`, if `activation` is not one of `None`, `"sigmoid"` or `"softmax"`, or if the configured baseline cannot be broadcast to the selected input shape.

Notes

`max_percentage_perturbed` determines the maximum portion of the input that can be replaced by the baseline. For example, a value of `0.5` limits the deletion process to half of the available features.

Setting `steps=-1` evaluates every possible number of deleted features between zero and the maximum number determined by `max_percentage_perturbed`.

The returned values are computed independently for each observation. They are not automatically aggregated into a dataset-level mean.

Applying `activation="softmax"` or `activation="sigmoid"` is useful when the desired interpretation is the drop in class probability rather than the drop in an arbitrary model score or logit.

class `xai_metrics.metrics.fidelity.completeness.insertion.Insertion`(*context*, *params=None*)

Bases: *BaseMetric*

Insertion fidelity metric.

This metric evaluates whether the features identified as important by an explanation are relevant to the model prediction. Starting from a baseline input, features are progressively restored to their original values following their attribution-based importance ranking.

For each observation, features with larger absolute attribution values are inserted first. After each insertion step, the model score is recomputed, producing an insertion curve that describes how the score evolves as increasingly important features are restored.

A faithful explanation is expected to identify features that strongly influence the model output. Consequently, inserting highly important features should produce a rapid increase in the model score and therefore a large area under the insertion curve. Higher values are better.

For each observation i , the procedure can be described as:

$$\begin{aligned} \mathbf{x}_i^{(0)} &= \mathbf{b}_i, \\ \mathbf{x}_i^{(k)} &= \text{insert}(\mathbf{b}_i, \mathbf{x}_i, M_i, k), \\ \text{score}_i^{(k)} &= g(f, \mathbf{x}_i^{(k)}, y_i) \end{aligned}$$

where f is the model, g is the scoring operator, $\mathbf{b}_i$ is the baseline input, M_i defines the feature ranking induced by the explanation and k is the number of restored features.

If the insertion curve contains $T + 1$ points, the reported score is its normalized trapezoidal area:

$$\text{Insertion}_i = \frac{1}{T} \sum_{t=0}^{T+1} \frac{s_i^{(t)} + s_i^{(t+1)}}{2}$$

A higher value indicates that inserting features considered important by the explanation causes the model score to increase more rapidly.

The implementation is based on the Insertion metric introduced by Petsiuk et al. (2018) and follows the implementation provided by Xplique, with support for PyTorch models, models exposing `predict` or `predict_proba`, custom scoring operators and configurable baseline values.

Petsiuk, V., Das, A., & Saenko, K. (2018). RISE: Randomized Input Sampling for Explanation of Black-box Models. In British Machine Vision Conference (BMVC).

Parameters

- **context** (`MetricContext`) – Shared metric evaluation context. It must contain the model, `X_test`, selected observations and attribution values. `y_test` may be `None` when the model score can be determined without explicit targets.
- **params** (`Mapping[str, Any]` or `None`, optional) – Metric-specific parameters. Supported keys are:
 - `batch_size` : int, optional Maximum number of perturbed observations processed at once when computing model scores. The default value is 64.
 - `baseline_mode` : float, array-like or callable, optional Baseline used as the initial state of the insertion process. A scalar value is broadcast to the input shape. If callable, it receives the selected input observations and must return a value or array broadcastable to their shape. The default value is `0.0`.
 - `steps` : int, optional Number of insertion intervals between the baseline and the maximum perturbation level. A value of -1 evaluates every possible number of inserted features up to the configured maximum. The default value is 10.
 - `max_percentage_perturbed` : float, optional Maximum fraction of input features that may be restored from the original input. Must belong to the interval `(0, 1]`. The default value is 1.0.
 - `operator` : callable or `None`, optional Custom scoring function `g(model, inputs, targets)` used to obtain one scalar score per observation. If `None`, the metric performs inference directly using the model and selects the corresponding target score. The default value is `None`.

- **activation** : {None, “sigmoid”, “softmax”}, optional Activation function applied to model predictions before selecting the score. None leaves predictions unchanged. The default value is None.

If None, an empty dictionary is used.

Notes

When no custom operator is provided, model inference is performed using, in order, a PyTorch `torch.nn.Module`, `predict_proba`, `predict` or direct model invocation.

For two-dimensional model outputs, targets are interpreted as class indices. If no targets are provided, the maximum model output for each observation is used.

Inputs and explanations are flattened before the insertion process. Therefore, each explanation must contain exactly the same number of feature values as its corresponding input observation.

Features are ranked by the absolute value of their attributions in descending order. Consequently, features with the largest attribution magnitude are restored first, independently of the attribution sign.

run()

Compute the Insertion metric.

The method selects the observations defined in the metric context and progressively restores their features from a configured baseline to their original values. Features are inserted in descending order of their absolute attribution values, so that the features considered most important by the explanation are restored first.

Inputs and explanations are flattened before ranking and perturbation. The insertion process starts from the baseline input. For each insertion step, the selected number of features is replaced by their corresponding values from the original input and the model score is recomputed.

The sequence of scores forms an insertion curve for each observation. The final score is the normalized trapezoidal area of that curve:

$$\text{Insertion}_i = \frac{1}{T} \sum_{t=0}^{T-1} \frac{s_i^{(t)} + s_i^{(t+1)}}{2}$$

where $s_i^{(t)}$ is the model score after restoring t features and T is the number of curve intervals.

Higher scores are better. A large value indicates that restoring features identified as important by the explanation rapidly increases the model score, suggesting that the explanation successfully captures features relevant to the prediction.

Returns

Insertion score for each evaluated observation. Higher values indicate better explanations.

Return type

List[float]

Raises

- **MetricSkipped** – If no observations are selected.
- **ValueError** – If the number of explanations differs from the number of selected observations, if inputs and explanations do not contain the same number of features per observation, if `batch_size` is not positive, if `max_percentage_perturbed` does not belong to `(0, 1]`, if `steps` is neither positive nor -1, if `activation` is not one of None, “sigmoid” or “softmax”, or if the configured baseline cannot be broadcast to the selected input shape.

Notes

`max_percentage_perturbed` determines the maximum portion of the original input that can be restored. For example, a value of `0.5` limits the insertion process to half of the available features.

Setting `steps=-1` evaluates every possible number of inserted features between zero and the maximum number determined by `max_percentage_perturbed`.

The returned values are computed independently for each observation. They are not automatically aggregated into a dataset-level mean.

Applying `activation="softmax"` or `activation="sigmoid"` is useful when the desired interpretation is the increase in class probability rather than the increase in an arbitrary model score or logit.

class `xai_metrics.metrics.fidelity.completeness.mufidelity.MuFidelity`(*context*, *params=None*)

Bases: `BaseMetric`

MuFidelity correlation metric.

This metric evaluates whether the importance assigned by an explanation to randomly selected subsets of features is consistent with the effect that perturbing those features has on the model output.

For each observation i , the metric generates R random masks. A mask value of one preserves a feature, while a value of zero replaces it with a baseline value. For every perturbation r , the perturbed input is:

$$\tilde{\mathbf{x}}_i^{(r)} = \mathbf{x}_i \odot \mathbf{m}_i^{(r)} + \mathbf{b}_i \odot (1 - \mathbf{m}_i^{(r)})$$

The prediction drop and the attribution sum associated with the perturbed features are:

$$\begin{aligned}\Delta_i^{(R)} &= g(f, \mathbf{x}_i, y_i) - g(f, \tilde{\mathbf{x}}_i^{(r)}, y_i), \\ A_i^{(r)} &= \sum_j \phi_{i,j} (1 - m_{i,j}^{(r)})\end{aligned}$$

The final MuFidelity score for observation i is the Spearman rank correlation between the prediction drops and attribution sums:

$$\text{MuFidelity}_i = \rho_{\text{Spearman}} \left(\{\Delta_i^{(r)}\}_{r=1}^R, \{A_i^{(r)}\}_{r=1}^R \right)$$

where f is the model, g is the scoring operator, $\mathbf{x}_i$ is the original input, $\mathbf{b}_i$ is the baseline input, $\phi_{i,j}$ is the attribution of the feature j and $\mathbf{m}_i^{(r)}$ is the sampled mask.

A high positive correlation indicates that subsets receiving larger attribution values tend to cause larger decreases in the model score when perturbed. Higher values are therefore better and indicate stronger agreement between the explanation and the model behaviour.

Random subsets are generated using Bernoulli masks. For tabular data, features are sampled independently. For time-series and image data, masks can be generated on a lower-resolution grid and expanded using nearest-neighbour interpolation, allowing groups of nearby features to be perturbed together.

The implementation is based on the fidelity correlation metric proposed by Bhatt et al. (2020) and follows the implementation provided by Xplique, with support for tabular data, time series, images, PyTorch models, models exposing `predict` or `predict_proba`, and custom scoring operators.

Bhatt, U., Weller, A., & Moura, J. M. F. (2020). Evaluating and Aggregating Feature-based Model Explanations. Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI).

Parameters

- **context** (`MetricContext`) – Shared metric evaluation context. It must contain the model, `X_test`, selected observations and attribution values. `y_test` may be `None` when the model score can be determined without explicit targets.

- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - **nb_samples** : int, optional Number of random perturbation masks generated for each observation. The default value is 200.
 - **n_masks** : int, optional Alias for **nb_samples**. It is used only when **nb_samples** is not provided.
 - **batch_size** : int or None, optional Maximum number of perturbed observations processed at once. If None, all perturbations for the selected observations may be processed together. The default value is 64.
 - **grid_size** : int or None, optional Resolution used to generate coarse perturbation masks. If None or zero-like, the size of the first feature dimension is used. For time series, masks are generated over (**grid_size**, **n_features**) and expanded over the temporal dimension. For images, masks are generated over (**grid_size**, **grid_size**) and expanded over the spatial dimensions.
 - **subset_percent** : float, optional Probability that each generated mask element is replaced by the baseline. The default value is 0.2.
 - **subset_probability** : float, optional Alias for **subset_percent**. It is used only when **subset_percent** is not provided.
 - **baseline_mode** : float, array-like or callable, optional Baseline used to replace perturbed features. If callable, it receives the repeated inputs associated with the generated perturbations. The default value is 0.0.
 - **operator** : callable or None, optional Custom scoring function $g(\text{model}, \text{inputs}, \text{targets})$ used to obtain one scalar score per observation. If None, the metric performs inference directly using the model and selects the corresponding target score. The default value is None.
 - **activation** : {None, “sigmoid”, “softmax”}, optional Activation function applied to model predictions before selecting the score. None leaves predictions unchanged. The default value is None.
 - **random_state** : int, np.random.Generator or None, optional Random seed or NumPy random generator used to create perturbation masks. Providing a fixed value makes mask generation reproducible. The default value is None.

If None, an empty dictionary is used.

Notes

When no custom **operator** is provided, model inference is performed using, in order, a PyTorch `torch.nn.Module`, `predict_proba`, `predict` or direct model invocation.

For two-dimensional model outputs, targets are interpreted as class indices. If no targets are provided, the maximum model output for each observation is used.

The effective processing batch size is divided between observations and perturbations so that several perturbations of several observations can be evaluated together without exceeding the configured **batch_size**.

The metric returns one correlation value per observation rather than automatically averaging the correlations across the dataset.

run()

Compute the MuFidelity correlation metric.

The method selects the observations defined in the metric context and first computes their original model scores. For each observation, multiple random subsets of features are then perturbed by replacing them with a configured baseline value.

For every perturbation, MuFidelity compares the model prediction drop with the sum of attributions assigned to the perturbed features:

$$\Delta_i^{(r)} = g(f, \mathbf{x}_i, y_i) + g(f, \tilde{\mathbf{x}}_i * (r), y_i),$$

$$A_i^{(r)} = \sum_j \phi_{i,j} \left(1 - m_{i,j}^{(r)}\right)$$

The reported score is:

$$\text{MuFidelity}_i = \rho_{\text{Spearman}} \left(\{\Delta_i^{(r)}\}_{r=1}^R, \{A_i^{(r)}\}_{r=1}^R \right)$$

A high positive correlation indicates that perturbing features assigned greater importance by the explanation tends to produce larger decreases in the model score. Higher values are therefore better.

Returns

MuFidelity correlation for each evaluated observation. Values are Spearman correlation coefficients and therefore normally lie in $[-1, 1]$. Higher positive values indicate stronger fidelity.

If either sequence involved in the correlation is constant and the Spearman coefficient is undefined, the score for that observation is set to 0.0 .

Return type

List[float]

Raises

- ***MetricSkipped*** – If no observations are selected.
- **ValueError** – If `batch_size` is not positive, if `activation` is not one of `None`, `"sigmoid"` or `"softmax"`, or if the number of explanations differs from the number of selected observations.

Notes

`nb_samples` controls the number of random subsets used to estimate the correlation. Larger values generally provide a more stable estimate at the cost of additional model evaluations.

`subset_percent` controls the probability that a feature or coarse mask element is replaced by the baseline in each perturbation.

When `grid_size` is smaller than the corresponding input dimension, groups of neighbouring features are perturbed together. This can be useful for medium- or high-dimensional inputs, particularly images and time series.

The returned values are computed independently for each observation. Unlike the original Xplique `evaluate` method, they are not automatically averaged into a single dataset-level fidelity score.

4.3.2 Soundness

```
class xai_metrics.metrics.fidelity.soundness.non_sensitivity.NonSensitivity(context,
                                                                           params=None)
```

Bases: *BaseMetric*

Quantus Non-Sensitivity metric.

This metric evaluates whether near-zero attribution values are assigned to features on which the model output is not functionally dependent. For each observation, Quantus compares features whose attribution is below a threshold with features whose perturbation produces only a negligible change in the selected model output.

The score measures disagreement between both sets of features. Lower values indicate better agreement between near-zero attributions and features that do not significantly affect the model output.

The metric is based on the Non-Sensitivity metric discussed by Nguyen and Rodríguez Martínez (2020), Ancona et al. (2019), and Montavon et al. (2018), as implemented in Quantus.

Parameters

- **context** (`MetricContext`) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.
- **params** (`Mapping[str, Any] or None, optional`) – Metric-specific parameters. Supported keys are:
 - `eps` : float, optional Threshold used to decide whether an attribution value is considered negligible and whether a prediction change is considered insignificant. The default value is `1e-5`.
 - `features_in_step` : int, optional Number of features perturbed at each step. The default value is `1`.
 - `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `True`.
 - `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `True`.
 - `perturb_baseline` : str, optional Baseline value used when perturbing features. Supported values depend on the Quantus perturbation function. Common values are "black", "white", "mean", "random" and "uniform". The default value is "black".

If `None`, an empty dictionary is used.

Notes

The wrapper uses the default functions provided by Quantus. The default perturbation function replaces selected features by the configured baseline, and the default aggregation function in Quantus is `numpy.mean`.

Quantus uses `eps` both to identify near-zero attributions and to decide whether the model output change after perturbation is negligible.

`run()`

Compute the Non-Sensitivity metric.

The method selects the observations defined in the metric context and passes their input data, labels and attribution values to `quantus.NonSensitivity`. Quantus perturbs groups of features, measures the resulting change in the selected model output and compares perturbation-insensitive features with features whose attribution values are below `eps`.

If all attribution values are negative, their treatment depends on the `abs` parameter. Their absolute values are used when `abs=True`; otherwise, the metric is skipped.

The model is set to evaluation mode before the metric is computed.

Returns

Non-Sensitivity score for each evaluated observation. Lower values indicate better agreement between near-zero attributions and features that do not significantly affect the model output.

Return type

List[float]

Raises***MetricSkipped*** – If all attribution values are negative and abs is False.

4.4 Robustness

class xai_metrics.metrics.robustness.average_stability.**AverageStability**(context, explain_func, params=None)

Bases: *BaseMetric*

Average Stability robustness metric.

This metric evaluates the sensitivity of an explanation to small perturbations of the input. For each observation, a set of neighbouring inputs is generated by adding random noise, their explanations are recomputed and the distance between each neighbouring explanation and the original explanation is measured.

For each observation i , the j -th neighbouring input is:

$$\tilde{\mathbf{x}}_i^{(j)} = \mathbf{x}_i + \boldsymbol{\varepsilon}^{(j)}$$

The distance between the original explanation and the neighbouring explanation is:

$$d_i^{(j)} = d\left(\phi(\tilde{\mathbf{x}}_i^{(j)}), \phi(\mathbf{x}_i)\right)$$

The Average Stability score for observation i is:

$$AS_i = \frac{1}{K} \sum_{j=1}^K d_i^{(j)}$$

where $\mathbf{x}_i$ is the original input, $\boldsymbol{\varepsilon}^{(j)}$ is the noise sampled for neighbour j , ϕ is the explanation function, d is the configured distance function and K is the number of sampled neighbours.

The metric returns one Average Stability value per observation. Lower values are better: a small score indicates that nearby inputs produce similar explanations, suggesting that the explanation method is locally stable.

The same set of random noise masks is reused for every evaluated observation, ensuring that all observations are evaluated using equivalent perturbation patterns.

The implementation is based on the Average Sensitivity or Stability metric discussed by Bhatt et al. (2020) and follows the implementation provided by Xplique, with support for custom explanation functions, configurable distance metrics and reproducible perturbation sampling.

Bhatt, U., Weller, A., & Moura, J. M. F. (2020). Evaluating and Aggregating Feature-based Model Explanations. Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI).

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, selected observations and attribution values. `y_test` may be `None` when the explanation function does not require explicit targets.
- **explain_func** (*ExplainFunc*) – Explanation function used to recompute attribution values for perturbed neighbouring inputs. It must accept the model, a batch of observations and their targets, and return one explanation per observation.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:

- `radius` : float, optional Maximum absolute magnitude of the random perturbation. The default value is `0.1`.
- `nb_samples` : int, optional Number of neighbouring inputs generated for each observation. The default value is `20`.
- `distance` : {"l1", "l2"} or callable, optional Distance used to compare explanations. "l1" computes the sum of absolute differences and "l2" computes the Euclidean distance. A custom callable must return a scalar. The default value is "l2".
- `random_state` : int, np.random.Generator or None, optional Random seed or NumPy random generator used for perturbation sampling. The default value is None.
- `symmetric_noise` : bool, optional Whether to sample perturbations from `[-radius, radius]` instead of `[0, radius]`. The default value is `False` in order to reproduce the behaviour of Xplique.

If None, an empty dictionary is used.

Raises

ValueError – If `explain_func` is None.

run()

Compute the Average Stability metric.

The method selects the observations defined in the metric context and creates `nb_samples` neighbouring inputs for each observation by adding random noise. The explanation function is evaluated for every neighbour, and each resulting explanation is compared with the original explanation.

For observation i , the final score is the mean explanation distance across its K sampled neighbours:

$$AS_i = \frac{1}{K} \sum_{j=1}^K d(\phi(\tilde{\mathbf{x}}_i^{(j)}), \phi(\mathbf{x}_i))$$

Lower values are better. A score close to zero indicates that small input perturbations produce explanations similar to the original one.

Returns

Average explanation distance for each evaluated observation. Lower values indicate more stable explanations.

Return type

List[float]

Raises

- **MetricSkipped** – If no observations are selected.
- **ValueError** – If the number or shape of the original explanations is incompatible with the selected inputs, if `radius` is negative, if `nb_samples` is not positive, if `distance` is invalid, if `explain_func` returns an invalid number or shape of explanations, or if a custom distance function does not return a scalar.

Notes

For `distance="l1"`, the distance between two explanations is computed as the sum of their absolute element-wise differences.

For `distance="l2"`, the Euclidean distance between two explanations is computed.

A custom distance callable receives the neighbouring explanation and the corresponding original explanation and must return a scalar value.

The returned values are computed independently for each observation. Unlike the original Xplique implementation, they are not automatically averaged into a single dataset-level stability score.

```
class xai_metrics.metrics.robustness.local_lipschitz_estimate.LocalLipschitzEstimate(context,
                                                                                     ex-
                                                                                     plain_func,
                                                                                     params=None)
```

Bases: *BaseMetric*

Quantus Local Lipschitz Estimate metric.

This metric evaluates the local stability of an explanation by comparing changes in the explanation with changes in the corresponding input. For each observation, Quantus generates neighbouring inputs using Gaussian noise, recomputes their explanations and calculates the ratio between the explanation distance and the input distance. The score is the maximum ratio obtained across the sampled neighbours.

Lower scores indicate that nearby inputs receive similar explanations and therefore correspond to more locally robust explanations.

The metric is based on the Local Lipschitz Estimate proposed by Alvarez-Melis and Jaakkola (2018) and implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations, attribution values and optional device information.
- **explain_func** (*ExplainFunc*) – Function used to generate explanations for perturbed inputs. The function must accept the model, a batch of inputs and optionally a batch of labels or targets, and must return a NumPy array containing the generated attributions.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - `nr_samples` : int, optional Number of perturbed samples generated for each observation. The default value is 200.
 - `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `False`.
 - `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `True`.
 - `perturb_mean` : float, optional Mean of the Gaussian noise used for perturbations. The default value is 0.0.
 - `perturb_std` : float, optional Standard deviation of the Gaussian noise used for perturbations. The default value is 0.1.

If `None`, an empty dictionary is used.

Notes

The wrapper uses the default functions provided by Quantus: the Lipschitz constant as the comparison function, Euclidean distance for the numerator and denominator, the default attribution normalisation function and Gaussian-noise perturbations.

The score is a finite-sample estimate based on the generated neighbours, rather than the exact maximum over the complete local neighbourhood.

Raises

ValueError – If `explain_func` is not provided.

Parameters

- **context** (*MetricContext*)
- **explain_func** (*ExplainFunc*)
- **params** (*Mapping[str, Any] | None*)

run()

Compute the Local Lipschitz Estimate metric.

The method passes the selected inputs, target labels, original attributions and explanation function to `quantus.LocalLipschitzEstimate`. Quantus samples neighbouring inputs, recomputes their explanations and returns the maximum ratio between explanation and input changes for each observation.

If all attribution values are negative, their absolute values are used when `abs=True`; otherwise, the metric is skipped. The model is set to training mode and the device stored in the context is forwarded to Quantus.

Returns

Local Lipschitz Estimate score for each evaluated observation. Lower values indicate greater local explanation stability.

Return type

List[float]

Raises

MetricSkipped – If all attribution values are negative and `abs` is `False`.

```
class xai_metrics.metrics.robustness.max_sensitivity.MaxSensitivity(context, explain_func,
                                                                    params=None)
```

Bases: *BaseMetric*

Quantus Max-Sensitivity metric.

This metric evaluates explanation robustness by measuring the largest relative change in an explanation under small input perturbations. For each observation, Quantus generates several perturbed inputs, recomputes their explanations and compares them with the original explanation. The score is the maximum ratio between the norm of the explanation change and the norm of the original explanation.

Lower scores indicate more robust explanations, while higher scores indicate greater sensitivity to input perturbations.

The metric is based on Max-Sensitivity proposed by Yeh et al. (2019) and discussed by Bhatt et al. (2020), as implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations, attribution values and optional device information.
- **explain_func** (*ExplainFunc*) – Function used to generate explanations for perturbed inputs. The function must be compatible with Quantus explanation functions and return a NumPy array containing the generated attributions.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - `nr_samples` : int, optional Number of perturbed samples generated for each observation. The default value is 200.
 - `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `False`.

- **normalise** : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `False`.
- **lower_bound** : float, optional Lower bound of the uniform noise used for perturbations. The default value is `0.2`.
- **upper_bound** : float or `None`, optional Upper bound of the uniform noise used for perturbations. If `None`, Quantus uses its default behaviour. The default value is `None`.

If `None`, an empty dictionary is used.

Notes

The wrapper uses the default functions provided by Quantus: element-wise difference to compare explanations, the Frobenius norm for the numerator and denominator, the default normalisation function and uniform-noise perturbations.

Raises

ValueError – If `explain_func` is not provided.

Parameters

- **context** (*MetricContext*)
- **explain_func** (*ExplainFunc*)
- **params** (*Mapping[str, Any] | None*)

run()

Compute the Max-Sensitivity metric.

The method passes the selected inputs, target labels, original attributions and explanation function to `quantus.MaxSensitivity`. Quantus repeatedly perturbs each input, recomputes its explanation and returns the maximum relative explanation change across the configured perturbations.

If all attribution values are negative, their treatment depends on the `abs` parameter. Their absolute values are used when `abs=True`; otherwise, the metric is skipped.

The model is set to training mode before the metric is computed. The device stored in the metric context is forwarded to Quantus.

Returns

Max-Sensitivity score for each evaluated observation. Lower values indicate explanations that are more robust to small random input perturbations.

Return type

List[float]

Raises

MetricSkipped – If all attribution values are negative and `abs` is `False`.

class `xai_metrics.metrics.robustness.mege.MeGe(context, training_func, explain_func, params=None)`

Bases: *BaseMetric*

MeGe representativity metric.

This metric evaluates the representativity and stability of explanations across models trained on different subsets of the available data. The main idea is that explanations produced by models with similar predictive behaviour should also be similar.

The selected observations are divided into `k_splits` equally sized subsets. For each split, a new model is trained using all remaining subsets. The excluded subset is passed to `training_func` as its validation or hold-out data.

Each trained model is then used to predict and explain every selected observation. Explanations produced by pairs of models are compared only when the observation belongs to a split excluded from the training data of at least one of the two models.

A comparison is considered valid when:

- at least one of the two models predicts the true class; and
- both models predict the same class.

For each valid comparison e_a and e_b , explanation similarity is measured using the Spearman-based distance:

$$d(e_a, e_b) = \sqrt{1 - |\rho_{\text{Spearman}}(e_a, e_b)|}$$

where ρ_{Spearman} is the Spearman rank correlation between the two explanations. Perfect positive and negative rank correlations therefore both produce a distance of zero.

For each observation i , let $\mathcal{D}_i$ be the set of valid pairwise explanation distances. Its MeGe scores is:

$$\text{MeGe}_i = \frac{1}{1 + \frac{1}{|\mathcal{D}_i|} \sum_{d \in \mathcal{D}_i} d}$$

where $\mathcal{D}_i$ is the set of valid explanation distances associated with that observation.

Higher values are better. A score close to 1 indicates that models with equivalent predictive behaviour produce highly similar explanations for the observation. Lower values indicate greater disagreement between explanations.

If no valid model pair is available for an observation, its score is returned as NaN. If no valid comparison is available for any selected observation, the metric is skipped.

The implementation is based on the MeGe representativity metric proposed by Fel et al. (2020) and follows the pairwise comparison strategy implemented in Xplique. Unlike the original Xplique implementation, this wrapper returns one MeGe score per observation and does not compute the ReCo consistency metric.

Fel, T., Vigouroux, D., Cadène, R., & Serre, T. (2020). How Good Is Your Explanation? Algorithmic Stability Measures to Assess the Quality of Explanations for Deep Neural Networks.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain `X_test`, `y_test` and the selected observations. Classification targets are required by this metric.
- **training_func** (*TrainingFunc*) – Function used to train one model for each data split. It must accept (`X_train`, `y_train`, `X_validation`, `y_validation`) and return a trained model compatible with the prediction procedure used by the metric.
- **explain_func** (*ExplainFunc*) – Explanation function used to compute attribution values for each trained model. It must accept (`model`, `inputs`, `targets`) and return one explanation per input observation.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - `k_splits` : int, optional Number of equally sized data splits used to train the model ensemble. For every split, one model is trained without that split. Must be at least 2 and exactly divide the number of selected observations. The default value is 4.

If None, an empty dictionary is used.

Raises

ValueError – If `training_func` or `explain_func` is None.

Notes

The metric trains `k_splits` independent models. Consequently, its computational cost can be substantially higher than metrics that only evaluate the model contained in the metric context.

The model stored in `context.model` is not used to compute the MeGe score itself. Models returned by `training_func` are used instead.

`run()`

Compute the MeGe representativity metric.

The selected observations are first divided into `k_splits` equally sized subsets. For each split, a model is trained on all remaining subsets using `training_func`. The excluded split is passed to the training function as validation or test data.

All trained models are then evaluated on every selected observation. Their predicted classes and explanations are computed and compared pairwise.

For a given pair of models and observation, the explanation distance is considered only when:

- the observation belongs to a split excluded from the training data of one of the two models;
- at least one model predicts the true class; and
- both models predict the same class.

For each valid pair, the explanation distance is:

$$d(e_a, e_b) = \sqrt{1 - |\rho_{\text{Spearman}}(e_a, e_b)|}$$

Let $\mathcal{D}_i$ be the valid distances collected for observation i . The final per-observation score is:

$$\text{MeGe}_i = \frac{1}{1 + \frac{1}{|\mathcal{D}_i|} \sum_{d \in \mathcal{D}_i} d}$$

Returns

MeGe score for each evaluated observation. Higher values indicate stronger agreement between explanations produced by models with the same predictive behaviour.

Observations for which no valid model pair is available receive NaN.

Return type

List[float]

Raises

- ***MetricSkipped*** – If classification targets are unavailable, if a trained model does not provide a supported prediction interface, if no observations are selected, or if no valid equal-prediction explanation pair is produced for any observation.
- ***ValueError*** – If `k_splits` is smaller than 2, if the number of selected observations is not divisible by `k_splits`, or if `explain_func` does not return one explanation for every selected observation.

Notes

Classification targets are required. Targets may be provided either as class labels or as two-dimensional encoded class vectors. In the latter case, the target class is obtained using `argmax`.

`k_splits` controls both the number of trained models and the number of possible pairwise explanation comparisons. Increasing it can provide more comparisons but also substantially increases the computational cost.

The returned scores are computed independently for each observation. This differs from the original Xplique implementation, where all valid equal-prediction distances are pooled into a single global MeGe score.

This implementation computes MeGe only. The ReCo consistency metric returned alongside MeGe by the original Xplique implementation is not calculated.

```
class xai_metrics.metrics.robustness.relative_input_stability.RelativeInputStability(context,
                                                                                       ex-
                                                                                       plain_func,
                                                                                       params=None)
```

Bases: *BaseMetric*

Quantus Relative Input Stability metric.

This metric evaluates explanation robustness by comparing the relative change in an explanation with the relative change in its input. For each observation, Quantus generates several perturbed inputs, recomputes their explanations and returns the maximum ratio between both relative changes.

Lower scores indicate more stable explanations, while higher scores indicate greater sensitivity to input perturbations.

The metric is based on Relative Input Stability proposed by Agarwal et al. (2022), as implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations, attribution values and optional device information.
- **explain_func** (*ExplainFunc*) – Function used to generate explanations for perturbed inputs. The function must be compatible with Quantus explanation functions and return a NumPy array containing the generated attributions.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - `nr_samples` : int, optional Number of perturbed samples generated for each observation. The default value is 200.
 - `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `False`.
 - `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `False`.

If `None`, an empty dictionary is used.

Notes

Quantus uses uniform-noise perturbations with an upper bound of 0.2. When normalisation is enabled, the default function is `normalise_by_average_second_moment_estimate`.

A constant of 1e-6 is used to avoid division by zero. By default, perturbations that change the model prediction produce nan scores.

Raises

ValueError – If `explain_func` is not provided.

Parameters

- **context** (*MetricContext*)
- **explain_func** (*ExplainFunc*)

- **params** (*Mapping[str, Any] | None*)

run()

Compute the Relative Input Stability metric.

The method passes the selected inputs, target labels, original attributions and explanation function to `quantus.RelativeInputStability`. Quantus perturbs each input, recomputes its explanation and returns the maximum ratio between the relative explanation change and the relative input change.

If all attribution values are negative, their treatment depends on the `abs` parameter. Their absolute values are used when `abs=True`; otherwise, the metric is skipped.

The model is set to evaluation mode before the metric is computed. The device stored in the metric context is forwarded to Quantus.

Returns

Relative Input Stability score for each evaluated observation. Lower values indicate more stable explanations. A score may be nan when a perturbation changes the model prediction.

Return type

`list[float]`

Raises

MetricSkipped – If all attribution values are negative and `abs` is `False`.

```
class xai_metrics.metrics.robustness.relative_output_stability.RelativeOutputStability(context,
                                                                                       ex-
                                                                                       plain_func,
                                                                                       params=None)
```

Bases: *BaseMetric*

Quantus Relative Output Stability metric.

This metric evaluates explanation robustness by comparing the relative change in an explanation with the corresponding change in the model output logits. For each observation, Quantus perturbs the input, recomputes its explanation and returns the maximum ratio obtained across the sampled perturbations.

Lower scores indicate that explanations change less relative to changes in the model output and are therefore considered more stable.

The metric is based on Relative Output Stability proposed by Agarwal et al. (2022), as implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations, attribution values and optional device information.
- **explain_func** (*ExplainFunc*) – Function used to generate explanations for perturbed inputs. The function must be compatible with Quantus explanation functions and return a NumPy array containing the generated attributions.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - `nr_samples` : int, optional Number of perturbed samples generated for each observation. The default value is 200.
 - `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `False`.
 - `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `False`.

If `None`, an empty dictionary is used.

Notes

Quantus uses uniform-noise perturbations with an upper bound of 0.2. When normalisation is enabled, the default function is `normalise_by_average_second_moment_estimate`.

A constant of 1e-6 is used to avoid division by zero. By default, perturbations that change the predicted class produce nan scores.

Raises

ValueError – If `explain_func` is not provided.

Parameters

- **context** (*MetricContext*)
- **explain_func** (*ExplainFunc*)
- **params** (*Mapping[str, Any] | None*)

run()

Compute the Relative Output Stability metric.

The method passes the selected inputs, target labels, original attributions and explanation function to `quantus.RelativeOutputStability`. Quantus repeatedly perturbs each input and returns the maximum ratio between the relative explanation change and the distance between the original and perturbed output-logit vectors.

If all attribution values are negative, their treatment depends on the `abs` parameter. Their absolute values are used when `abs=True`; otherwise, the metric is skipped.

The model is set to evaluation mode before the metric is computed. The device stored in the metric context is forwarded to Quantus.

Returns

Relative Output Stability score for each evaluated observation. Lower values indicate more stable explanations. A score may be `nan` when a perturbation changes the predicted class.

Return type

`list[float]`

Raises

MetricSkipped – If all attribution values are negative and `abs` is `False`.

4.5 Sensitivity

```
class xai_metrics.metrics.sensitivity.avg_sensitivity.AvgSensitivity(context, explain_func,
                                                                    params=None)
```

Bases: *BaseMetric*

Quantus Average Sensitivity metric.

This metric evaluates explanation robustness by measuring the average relative change in an explanation when small random perturbations are applied to its input. For each observation, Quantus generates several perturbed inputs, recomputes their explanations using `explain_func` and compares them with the original explanation.

The sensitivity of each perturbation is computed from the norm of the explanation difference relative to the norm of the original explanation. The final score is the average over all sampled perturbations.

Lower scores indicate more robust explanations, whereas higher scores indicate greater sensitivity to small input changes.

The metric is based on Average Sensitivity proposed by Yeh et al. (2019) and discussed by Bhatt et al. (2020), as implemented in Quantus.

Initialize the Average Sensitivity metric.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations, attribution values and optional device information.
- **explain_func** (*ExplainFunc*) – Function used to generate explanations for perturbed inputs. The function must be compatible with Quantus explanation functions and return a NumPy array containing the generated attributions.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - `nr_samples` : int, optional Number of perturbed samples generated for each observation. The default value is 200.
 - `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `False`.
 - `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `False`.
 - `lower_bound` : float, optional Lower bound of the uniform noise used for perturbations. The default value is 0.2.
 - `upper_bound` : float or None, optional Upper bound of the uniform noise used for perturbations. If None, Quantus uses its default behaviour. The default value is None.

If None, an empty dictionary is used.

Notes

The wrapper uses the default functions provided by Quantus: element-wise difference for comparing explanations, the Frobenius norm for the numerator and denominator, the default normalisation function and uniform-noise perturbations.

Raises

ValueError – If `explain_func` is not provided.

Parameters

- **context** (*MetricContext*)
- **explain_func** (*ExplainFunc*)
- **params** (*Mapping[str, Any] | None*)

run()

Compute the Average Sensitivity metric.

The method passes the selected inputs, target labels, original attributions and explanation function to `quantus.AvgSensitivity`. Quantus repeatedly perturbs each input, recomputes its explanation and averages the relative explanation changes across the configured number of samples.

If all attribution values are negative, their absolute values are used when `abs=True`; otherwise, the metric is skipped. The model is set to training mode and the device stored in the context is forwarded to Quantus.

Returns

Average Sensitivity score for each evaluated observation. Lower values indicate greater robustness to random input perturbations.

Return type

List[float]

Raises

MetricSkipped – If all attribution values are negative and `abs` is `False`.

```
class xai_metrics.metrics.sensitivity.model_randomization.ModelRandomization(context,
                                                                              explain_func,
                                                                              params=None)
```

Bases: *BaseMetric*

Model Randomization sensitivity metric.

This metric evaluates whether an explanation depends on the parameters learned by the model. A copy of the original model is created, a fraction of its parameterised layers is randomized, and explanations are recomputed using the randomized model.

For each observation i , the metric compares the original explanation with the explanation produced by the randomized model:

$$MR_i = \rho_{\text{Spearman}} \left(\phi(f, x_i, y_i), \phi(\tilde{f}, x_i, y_i) \right)$$

where f is the original model, $\tilde{f}$ is its randomized copy and ϕ is the explanation function.

A low correlation indicates that the explanation changes when learned model parameters are replaced, which is the expected behaviour for an explanation method that depends on the model. A high positive correlation indicates that the attribution ranking remains similar despite parameter randomization.

The metric returns one Spearman correlation value per observation. Values normally lie in $[-1, 1]$. Values close to 1 indicate very similar attribution rankings, values close to 0 indicate weak rank association, and negative values indicate reversed attribution rankings.

The original model stored in the metric context is never modified. Randomization if applied exclusively to a deep copy. Parameter values in selected layers are independently sampled from a uniform distribution over $[0, 1]$.

The implementation is based on the Model Randomization sanity check proposed by Adebayo et al. (2018) and follows the implementation provided by Xplique, adapted to PyTorch models and per-observation outputs.

Adebayo, J., Gilmer, J., Muelly, M., Goodfellow, I., Hardt, M., & Kim, B. (2018). Sanity Checks for Saliency Maps. *Advances in Neural Information Processing Systems*, 31.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain a PyTorch model, `X_test`, selected observations and attribution values. `y_test` may be `None` when the explanation function does not require explicit targets.
- **explain_func** (*ExplainFunc*) – Explanation function used to recompute attribution values with the randomized model. It must accept (`model`, `inputs`, `targets`) and return one explanation per input observation.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - `batch_size` : int or `None`, optional Maximum number of observations whose explanations are recomputed at once. If `None`, all selected observations are processed together. The default value is 64.

- **fraction** : float, optional Fraction of parameterised model layers to randomize. Must belong to the interval $[0, 1]$. The number of randomized layers is computed as `int(number_of_layers * fraction)`. The default value is 0.25.
- **reverse** : bool, optional Whether to reverse the order of parameterised layers before selecting those to randomize. If True, randomization starts from the last parameterised layers encountered in the model. The default value is True.
- **random_state** : int or None, optional Random seed used to generate randomized parameter values. The default value is 42.

If None, an empty dictionary is used.

Raises

ValueError – If `explain_func` is None.

Notes

This implementation currently supports PyTorch models only.

The original attribution values stored in the metric context are used as reference explanations. Only explanations for the randomized model are recomputed using `explain_func`.

Randomization is applied to a deep copy of the model, so the model stored in the metric context is not modified.

`run()`

Compute the Model Randomization metric.

The method first creates a deep copy of the PyTorch model contained in the metric context. Parameterised layers are collected and optionally traversed in reverse order. A configurable fraction of these layers is then randomized by replacing their parameter values with samples drawn independently from a uniform distribution over $[0, 1]$.

The explanation function is evaluated using the randomized model for all selected observations. For each observation i , its stored original explanation is compared with the randomized explanation:

$$MR_i = \rho_{\text{Spearman}} \left(\phi(f, x_i, y_i) \phi(\tilde{f}, x_i, y_i) \right)$$

where f is the original model and $\tilde{f}$ is its randomized copy.

Returns

Spearman correlation between the original and randomized explanation for each evaluated observation. Values normally lie in $[-1, 1]$.

Values close to 1 indicate highly similar attribution rankings, values close to 0 indicate weak rank association, and negative values indicate reversed rankings.

Return type

List[float]

Raises

- ***MetricSkipped*** – If the model is not a PyTorch `torch.nn.Module`, if no observations are selected, or if the model contains no parameterised layers.
- **ValueError** – If the number of original explanations differs from the number of selected observations, if `batch_size` is not positive, if `fraction` does not belong to $[0, 1]$, or if `explain_func` returns explanations whose shape differs from the original attribution shape.

Notes

The number of randomized layers is calculated as:

$$\lfloor \text{number_of_layers} \times \text{fraction} \rfloor$$

Therefore, a small positive `fraction` can select zero layers for a small model.

When `reverse=True`, the order of the collected layers is reversed before selecting the layers to randomize. This approximates progressive randomization beginning from the output side of sequential models.

The original model is not modified because all parameter changes are applied to a deep copy. This implementation returns one score per observation rather than a single dataset-level aggregate.

```
class xai_metrics.metrics.sensitivity.random_logit.RandomLogit(context, explain_func,
                                                         params=None)
```

Bases: [BaseMetric](#)

Random Logit sensitivity metric.

This metric evaluates whether an explanation depends on the target class for which it is computed. For each observation, the original explanation for the true target class is compared with an explanation generated for a randomly selected alternative class.

For each observation i with target class y_i , an alternative class $\tilde{y}_i$ is sampled uniformly from all available classes:

$$\tilde{y}_i \sim \text{Uniform}(\{0, \dots, C - 1\} \setminus \{y_i\})$$

The Random Logit score is the global Structural Similarity Index between the original and alternative-target explanations:

$$\text{RL}_i = \text{SSIM}(\phi(f, x_i, y_i), \phi(f, x_i, \tilde{y}_i))$$

where f is the model and ϕ is the explanation function.

Lower values indicate that the explanation changes when the target class changes. This suggests greater class specificity. High similarity indicates that explanations remain similar across target classes.

The metric returns one similarity value per observation. Lower values indicate greater sensitivity to the target class, meaning that the explanation changes when a different class is explained. High similarity indicates that the explanation remains similar across different target classes, which may suggest insufficient class specificity.

The number of classes can either be provided explicitly or inferred from a PyTorch classification model or a model exposing `predict_proba`. Both integer class labels and two-dimensional encoded class targets are supported.

The implementation is based on the Random Logit sanity check inspired by Adebayo et al. (2018) and follows the implementation provided by Xplique, adapted to per-observation outputs and NumPy-based SSIM computation.

Adebayo, J., Gilmer, J., Muelly, M., Goodfellow, I., Hardt, M., & Kim, B. (2018). Sanity Checks for Saliency Maps. *Advances in Neural Information Processing Systems*, 31.

Parameters

- **context** ([MetricContext](#)) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values. Classification targets are required by this metric.
- **explain_func** ([ExplainFunc](#)) – Explanation function used to recompute attribution values for alternative target classes. It must accept (`model`, `inputs`, `targets`) and return one explanation per input observation with the same shape as the original attributions.
- **params** ([Mapping\[str, Any\]](#) or `None`, optional) – Metric-specific parameters. Supported keys are:

- `batch_size` : int or None, optional Maximum number of observations processed at once. If None, all selected observations are processed together. The default value is 64.
- `num_classes` : int or None, optional Number of output classes. If provided, it must be at least 2. If None, the number of classes is inferred from the model output. The default value is None.
- `random_state` : int, np.random.Generator or None, optional Random seed or NumPy random generator used to sample alternative target classes. The default value is 42.

If None, an empty dictionary is used.

Raises

ValueError – If `explain_func` is None.

Notes

Original attribution values stored in the metric context are used as reference explanations. Only explanations for randomly selected alternative classes are recomputed.

For two-dimensional targets, the target class is obtained using `argmax` and alternative targets are passed to `explain_func` using one-hot encoding. For one-dimensional targets, alternative targets are passed as integer class indices.

run()

Compute the Random Logit metric.

For every selected observation, the method samples an alternative target class that differs from its true target. With C output classes, every eligible alternative class has probability $1/(C - 1)$.

The explanation function is evaluated using these alternative targets. The stored original explanation and the alternative-target explanation are then compared using global SSIM:

$$RL_i = \text{SSIM}(\phi(f, x_i, y_i), \phi(f, x_i, \tilde{y}_i))$$

where $\tilde{y}_i \neq y_i$.

Lower similarity values indicate that the explanation changes when the target class changes and therefore suggest greater target sensitivity or class specificity. High similarity values indicate that the explanation remains similar across different target classes.

Returns

SSIM similarity between the original target explanation and the randomly selected alternative-target explanation for each evaluated observation.

Lower values indicate greater sensitivity to the target class.

Return type

List[float]

Raises

- **MetricSkipped** – If no observations are selected, if classification targets are not available, if the number of classes cannot be determined, or if the model does not represent a classification problem with at least two output classes.
- **ValueError** – If the number of explanations differs from the number of selected observations, if `batch_size` is not positive, if `num_classes` is smaller than 2, if a target class lies outside the configured model output range, or if `explain_func` returns explanations with a shape different from the original attributions.

Notes

If targets are two-dimensional, they are interpreted as encoded class vectors and converted to class indices using `argmax`. Alternative targets are then passed to `explain_func` as one-hot encoded vectors.

If targets are one-dimensional, they are interpreted as integer class indices and alternative targets are passed using the same representation.

A single alternative class is sampled independently for each observation. Consequently, the score can vary across executions unless `random_state` is fixed.

Unlike the original Xplique `evaluate` method, this implementation returns one similarity value per observation rather than automatically averaging the results into a single dataset-level score.

PYTHON MODULE INDEX

X

`xai_metrics.base.base_explainer`, 2
`xai_metrics.base.base_metric`, 1
`xai_metrics.base.explainer_registry`, 4
`xai_metrics.base.metric_registry`, 3
`xai_metrics.config.config_controller`, 7
`xai_metrics.explainers.breakdown`, 15
`xai_metrics.explainers.lime`, 16
`xai_metrics.explainers.maple`, 18
`xai_metrics.explainers.shap`, 19
`xai_metrics.metrics.complexity.complexity_metric`, 21
`xai_metrics.metrics.complexity.sparseness`, 22
`xai_metrics.metrics.faithfulness.consistency`, 23
`xai_metrics.metrics.faithfulness.faithfulness`, 23
`xai_metrics.metrics.faithfulness.faithfulness_estimate`, 25
`xai_metrics.metrics.faithfulness.monotonicity`, 26
`xai_metrics.metrics.faithfulness.monotonicity_correlation`, 27
`xai_metrics.metrics.faithfulness.monotonicity_metric`, 28
`xai_metrics.metrics.faithfulness.sensitivity_n`, 29
`xai_metrics.metrics.faithfulness.sufficiency`, 30
`xai_metrics.metrics.fidelity.completeness.average_drop`, 31
`xai_metrics.metrics.fidelity.completeness.average_gain`, 33
`xai_metrics.metrics.fidelity.completeness.average_increase`, 35
`xai_metrics.metrics.fidelity.completeness.completeness_metric`, 37
`xai_metrics.metrics.fidelity.completeness.deletion`, 38
`xai_metrics.metrics.fidelity.completeness.insertion`, 40
`xai_metrics.metrics.fidelity.completeness.mufidelity`, 43
`xai_metrics.metrics.fidelity.soundness.non_sensitivity`, 45
`xai_metrics.metrics.robustness.average_stability`, 47
`xai_metrics.metrics.robustness.local_lipschitz_estimate`, 49
`xai_metrics.metrics.robustness.max_sensitivity`, 50
`xai_metrics.metrics.robustness.mege`, 51
`xai_metrics.metrics.robustness.relative_input_stability`, 54
`xai_metrics.metrics.robustness.relative_output_stability`, 55
`xai_metrics.metrics.sensitivity.avg_sensitivity`, 56
`xai_metrics.metrics.sensitivity.model_randomization`, 58
`xai_metrics.metrics.sensitivity.random_logit`, 60
`xai_metrics.reporting.reporting`, 12
`xai_metrics.runner.runner`, 9

INDEX

A

`as_explain_func()` (`xai_metrics.base.base_explainer.BaseExplainer` module method), 2

`AverageDrop` (class in `xai_metrics.metrics.fidelity.completeness.average_drop`), 31

`AverageGain` (class in `xai_metrics.metrics.fidelity.completeness.average_gain`), 33

`AverageIncrease` (class in `xai_metrics.metrics.fidelity.completeness.average_increase`), 35

`AverageStability` (class in `xai_metrics.metrics.robustness.average_stability`), 47

`AvgSensitivity` (class in `xai_metrics.metrics.sensitivity.avg_sensitivity`), 56

B

`BaseExplainer` (class in `xai_metrics.base.base_explainer`), 2

`BaseMetric` (class in `xai_metrics.base.base_metric`), 1

`BreakDownExplainer` (class in `xai_metrics.explainers.breakdown`), 15

`build_explainers_context()` (`xai_metrics.config.config_controller.ConfigController` method), 7

`build_explainers_contexts()` (`xai_metrics.config.config_controller.ConfigController` method), 8

`build_explainers_from_config()` (in module `xai_metrics.base.explainer_registry`), 4

`build_metric_context()` (`xai_metrics.config.config_controller.ConfigController` method), 8

`build_metric_contexts()` (`xai_metrics.config.config_controller.ConfigController` method), 8

`build_metrics_from_config()` (in module `xai_metrics.base.metric_registry`), 3

`build_observation_reports()` (in module `xai_metrics.reporting.reporting`), 12

`build_reports()` (in module `xai_metrics.reporting.reporting`), 12

C

`Completeness` (class in `xai_metrics.metrics.fidelity.completeness.completeness_metric`), 37

`Complexity` (class in `xai_metrics.metrics.complexity.complexity_metric`), 21

`ConfigController` (class in `xai_metrics.config.config_controller`), 7

`Consistency` (class in `xai_metrics.metrics.faithfulness.consistency`), 23

D

`default_model_loader()` (in module `xai_metrics.config.config_controller`), 9

`Deletion` (class in `xai_metrics.metrics.fidelity.completeness.deletion`), 38

E

`explain()` (`xai_metrics.base.base_explainer.BaseExplainer` method), 2

`explain()` (`xai_metrics.explainers.breakdown.BreakDownExplainer` method), 16

`explain()` (`xai_metrics.explainers.lime.LIMEExplainer` method), 17

`explain()` (`xai_metrics.explainers.maple.MAPLEExplainer` method), 19

`explain()` (`xai_metrics.explainers.shap.SHAPEExplainer` method), 20

`ExplainerContext` (class in `xai_metrics.base.base_explainer`), 3

`ExplainerSkipped`, 3

F

`Faithfulness` (class in `xai_metrics.metrics.faithfulness.faithfulness`), 23

FaithfulnessEstimate	(class in xai_metrics.metrics.faithfulness.consistency, xai_metrics.metrics.faithfulness.faithfulness_estimate), 25	23
G		
get_explainers_config()	(xai_metrics.config.config_controller.ConfigController method), 8	25
get_metrics_config()	(xai_metrics.config.config_controller.ConfigController method), 9	27
I		
Insertion	(class in xai_metrics.metrics.fidelity.completeness.insertion), 40	29
L		
LIMEExplainer	(class in xai_metrics.explainers.lime), 16	31
list_explainers()	(in module xai_metrics.base.explainer_registry), 5	33
list_metrics()	(in module xai_metrics.base.metric_registry), 4	35
LocalLipschitzEstimate	(class in xai_metrics.metrics.robustness.local_lipschitz_estimate), 49	37
M		
MAPLEExplainer	(class in xai_metrics.explainers.maple), 18	38
MaxSensitivity	(class in xai_metrics.metrics.robustness.max_sensitivity), 50	40
MeGe	(class in xai_metrics.metrics.robustness.mege), 51	43
MetricContext	(class in xai_metrics.base.base_metric), 1	45
MetricSkipped	2	47
ModelRandomization	(class in xai_metrics.metrics.sensitivity.model_randomization), 58	49
module		50
xai_metrics.base.base_explainer	2	51
xai_metrics.base.base_metric	1	54
xai_metrics.base.explainer_registry	4	55
xai_metrics.base.metric_registry	3	56
xai_metrics.config.config_controller	7	58
xai_metrics.explainers.breakdown	15	60
xai_metrics.explainers.lime	16	
xai_metrics.explainers.maple	18	
xai_metrics.explainers.shap	19	
xai_metrics.metrics.complexity.complexity_metric	21	
xai_metrics.metrics.complexity.sparseness	22	
Monotonicity	(class in xai_metrics.metrics.faithfulness.monotonicity), 26	
xai_metrics.metrics.faithfulness.consistency		23
xai_metrics.metrics.faithfulness.faithfulness		23
xai_metrics.metrics.faithfulness.faithfulness_estimate		25
xai_metrics.metrics.faithfulness.monotonicity		26
xai_metrics.metrics.faithfulness.monotonicity_correlation		27
xai_metrics.metrics.faithfulness.monotonicity_metric		28
xai_metrics.metrics.faithfulness.sensitivity_n		29
xai_metrics.metrics.faithfulness.sufficiency		30
xai_metrics.metrics.fidelity.completeness.average_drop		31
xai_metrics.metrics.fidelity.completeness.average_gain		33
xai_metrics.metrics.fidelity.completeness.average_increment		35
xai_metrics.metrics.fidelity.completeness.completeness		37
xai_metrics.metrics.fidelity.completeness.deletion		38
xai_metrics.metrics.fidelity.completeness.insertion		40
xai_metrics.metrics.fidelity.completeness.mufidelity		43
xai_metrics.metrics.fidelity.soundness.non_sensitivity		45
xai_metrics.metrics.robustness.average_stability		47
xai_metrics.metrics.robustness.local_lipschitz_estimate		49
xai_metrics.metrics.robustness.max_sensitivity		50
xai_metrics.metrics.robustness.mege	51	
xai_metrics.metrics.robustness.relative_input_stability		54
xai_metrics.metrics.robustness.relative_output_stability		55
xai_metrics.metrics.sensitivity.avg_sensitivity		56
xai_metrics.metrics.sensitivity.model_randomization		58
xai_metrics.metrics.sensitivity.random_logit		60
xai_metrics.reporting.reporting	12	
xai_metrics.runner.runner	9	

module, 1	module, 49
xai_metrics.base.explainer_registry	xai_metrics.metrics.robustness.max_sensitivity
module, 4	module, 50
xai_metrics.base.metric_registry	xai_metrics.metrics.robustness.mege
module, 3	module, 51
xai_metrics.config.config_controller	xai_metrics.metrics.robustness.relative_input_stability
module, 7	module, 54
xai_metrics.explainers.breakdown	xai_metrics.metrics.robustness.relative_output_stability
module, 15	module, 55
xai_metrics.explainers.lime	xai_metrics.metrics.sensitivity.avg_sensitivity
module, 16	module, 56
xai_metrics.explainers.maple	xai_metrics.metrics.sensitivity.model_randomization
module, 18	module, 58
xai_metrics.explainers.shap	xai_metrics.metrics.sensitivity.random_logit
module, 19	module, 60
xai_metrics.metrics.complexity.complexity_metric	xai_metrics.reporting.reporting
module, 21	module, 12
xai_metrics.metrics.complexity.sparseness	xai_metrics.runner.runner
module, 22	module, 9
xai_metrics.metrics.faithfulness.consistency	
module, 23	
xai_metrics.metrics.faithfulness.faithfulness	
module, 23	
xai_metrics.metrics.faithfulness.faithfulness_estimate	
module, 25	
xai_metrics.metrics.faithfulness.monotonicity	
module, 26	
xai_metrics.metrics.faithfulness.monotonicity_correlation	
module, 27	
xai_metrics.metrics.faithfulness.monotonicity_metric	
module, 28	
xai_metrics.metrics.faithfulness.sensitivity_n	
module, 29	
xai_metrics.metrics.faithfulness.sufficiency	
module, 30	
xai_metrics.metrics.fidelity.completeness.average_drop	
module, 31	
xai_metrics.metrics.fidelity.completeness.average_gain	
module, 33	
xai_metrics.metrics.fidelity.completeness.average_increase	
module, 35	
xai_metrics.metrics.fidelity.completeness.completeness_metric	
module, 37	
xai_metrics.metrics.fidelity.completeness.deletion	
module, 38	
xai_metrics.metrics.fidelity.completeness.insertion	
module, 40	
xai_metrics.metrics.fidelity.completeness.mufidelity	
module, 43	
xai_metrics.metrics.fidelity.soundness.non_sensitivity	
module, 45	
xai_metrics.metrics.robustness.average_stability	
module, 47	
xai_metrics.metrics.robustness.local_lipschitz_estimate	